Compound AI systems & Agents

CS 5624: Natural Language Processing

Fall 2026

https://tuvllms.github.io/nlp-fall-2026/

Tu Vu

VIRGINIA TECH

Logistics

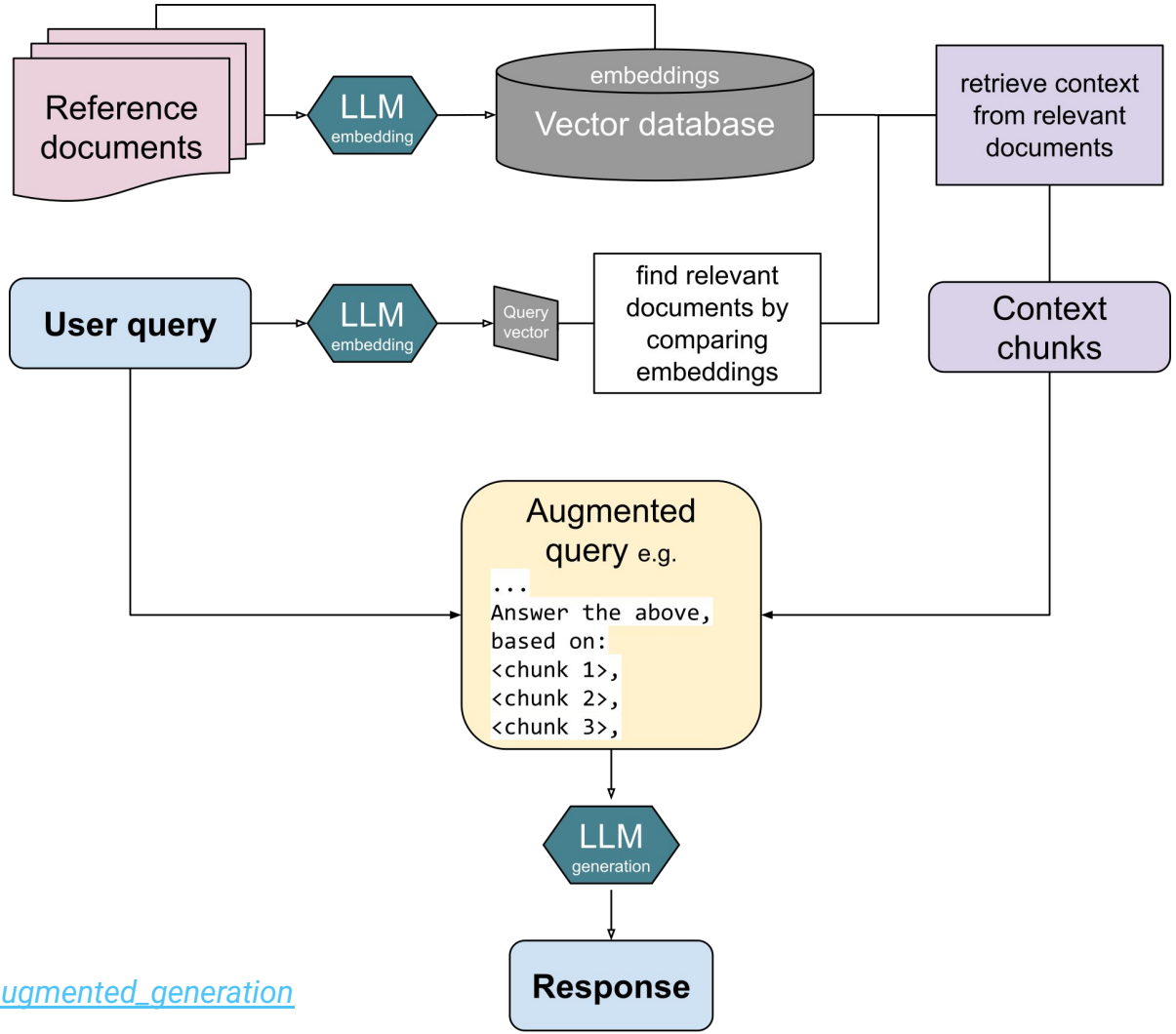
- HW2 due this Friday 10/13
- Proposal grades & feedback releasing today
- HW3, Quiz3 coming

AI news

Retrieval-Augmented Generation (RAG)

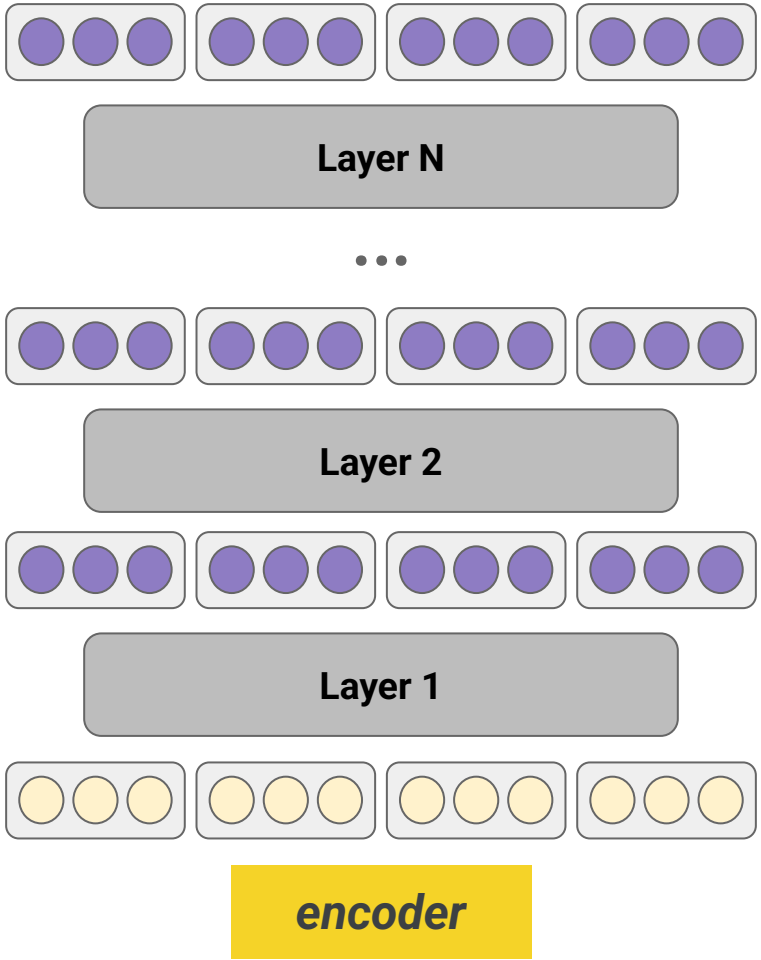
- Vanilla RAG
  - E.g., RAG, REALM
- RAG++
  - E.g., FreshLLMs, ReAct, Toolformer
- RAG + reasoning, agentic RAG, agentic memory
  - E.g., Self-RAG, Search-R1, Deep Research, A-Mem

Vanilla RAG

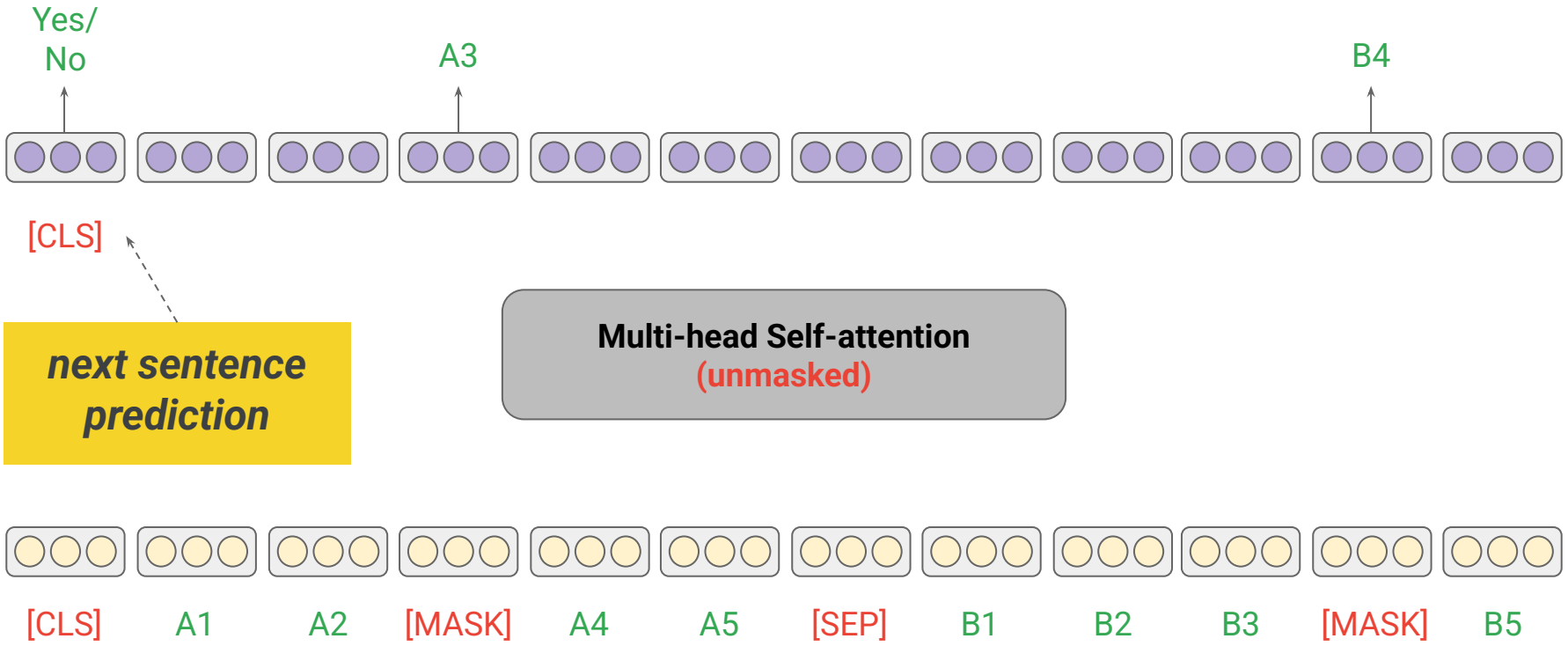


https://en.wikipedia.org/wiki/Retrieval-augmented_generation

Encoder (N layers)



BERT encoder



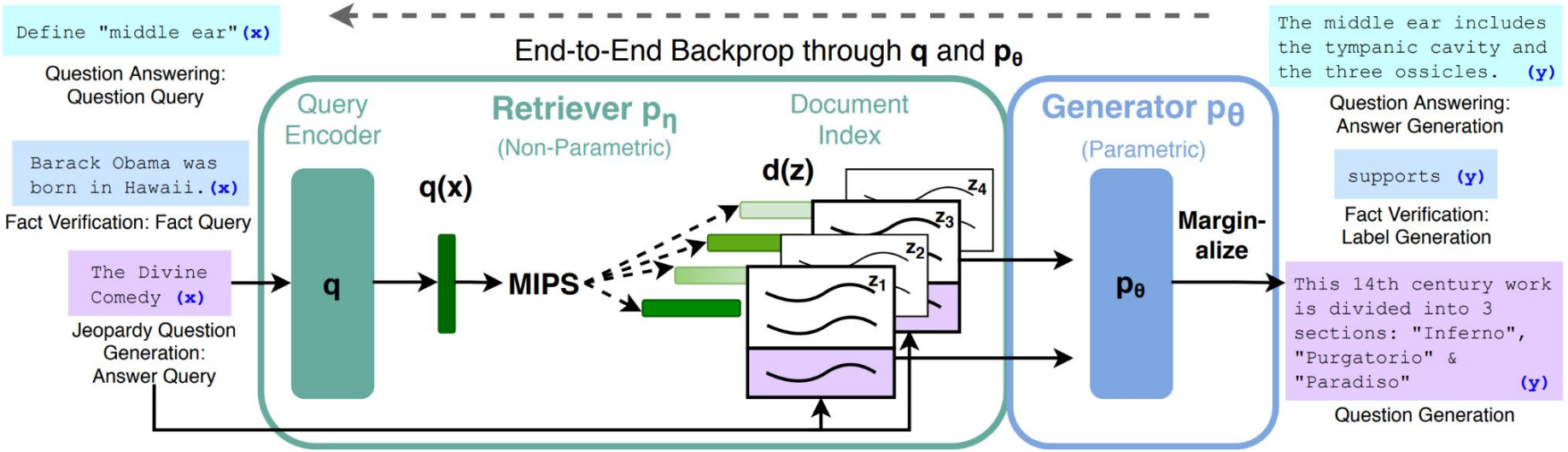
Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Patrick Lewis[†‡], Ethan Perez[⋆],

Aleksandra Piktus[†], Fabio Petroni[†], Vladimir Karpukhin[†], Naman Goyal[†], Heinrich Küttler[†],

Mike Lewis[†], Wen-tau Yih[†], Tim Rocktäschel[†‡], Sebastian Riedel[†‡], Douwe Kiela[†]

[†]Facebook AI Research; [‡]University College London; [⋆]New York University;
plewis@fb.com


Define "middle ear" (x)
Question Answering: Question Query
Barack Obama was born in Hawaii. (x)
Fact Verification: Fact Query
The Divine Comedy (x)
Jeopardy Question Generation: Answer Query
End-to-End Backprop through q and pθ
Query Encoder
q
q(x)
Retriever pη
(Non-Parametric)
MIPS
Document Index
d(z)
z1
z2
z3
z4
Generator pθ
(Parametric)
pθ
Margin-alize
The middle ear includes the tympanic cavity and the three ossicles. (y)
Question Answering: Answer Generation
supports (y)
Fact Verification: Label Generation
This 14th century work is divided into 3 sections: "Inferno", "Purgatorio" & "Paradiso" (y)
Question Generation

Component	Type	Role	Trainable?
Index	Data structure	Stores document embeddings for nearest-neighbor search	No
Retriever	Model	Computes similarity between query and documents; retrieves top-K	Yes (query encoder usually)
Query encoder	Neural network	Encodes the input text into an embedding	Yes
Document encoder	Neural network	Encodes documents into embeddings used to build the index	Sometimes frozen
Generator	Seq2seq model	Generates text conditioned on the input and retrieved docs	Yes

Dense Passage Retrieval for Open-Domain Question Answering

Vladimir Karpukhin[*], **Barlas Oğuz**[*], **Sewon Min**[†], **Patrick Lewis**,
Ledell Wu, **Sergey Edunov**, **Danqi Chen**[‡], **Wen-tau Yih**
Facebook AI [†]University of Washington [‡]Princeton University
{vladk, barlaso, plewis, ledell, edunov, scottyih}@fb.com
sewon@cs.washington.edu
danqic@cs.princeton.edu

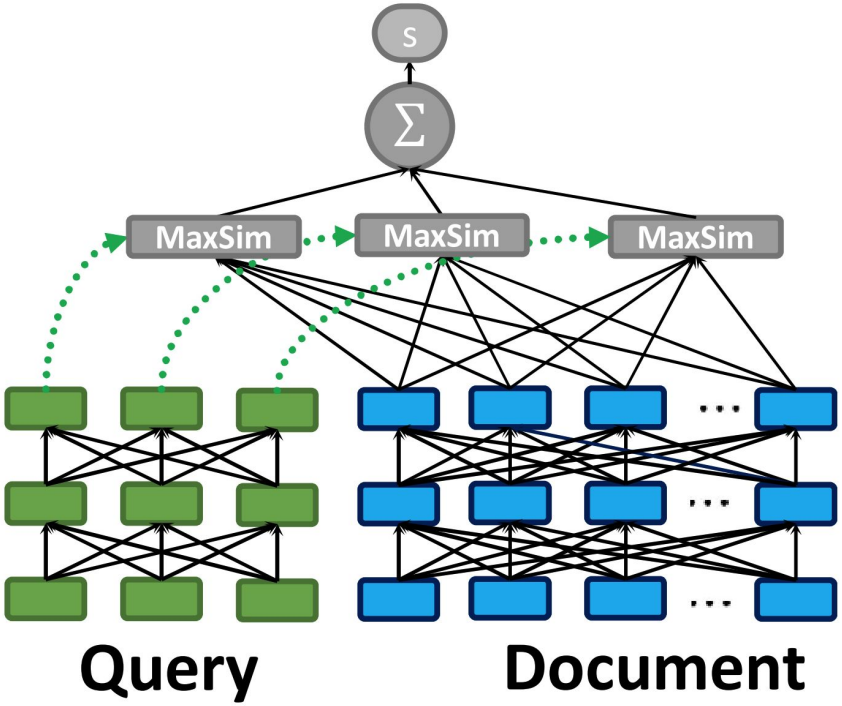
Dense Passage Retrieval (DPR)

Let $\mathcal{D} = \{(q_i, p_i^+, p_{i,1}^-, \ldots, p_{i,n_i}^-)\}_{i=1}^m$ denote the training set that contains m instances. Each instance includes a question q_i, one relevant (positive) passage p_i^+, and n_i irrelevant (negative) passages $p_{i,j}^-$.

The loss function is optimized as the negative log-likelihood of the positive passage:

$$\mathcal{L}(q_i, p_i^+, p_{i,1}^-, \ldots, p_{i,n_i}^-) = -\log \frac{\exp\big(\mathrm{sim}(q_i, p_i^+)\big)}{\exp\big(\mathrm{sim}(q_i, p_i^+)\big) + \sum_{j=1}^{n_i} \exp\big(\mathrm{sim}(q_i, p_{i,j}^-)\big)}.$$

ColBert's late interaction



(d) Late Interaction

(i.e., the proposed ColBERT)

https://arxiv.org/abs/2004.12832

Google DeepMind

On the Theoretical Limitations of Embedding-Based Retrieval

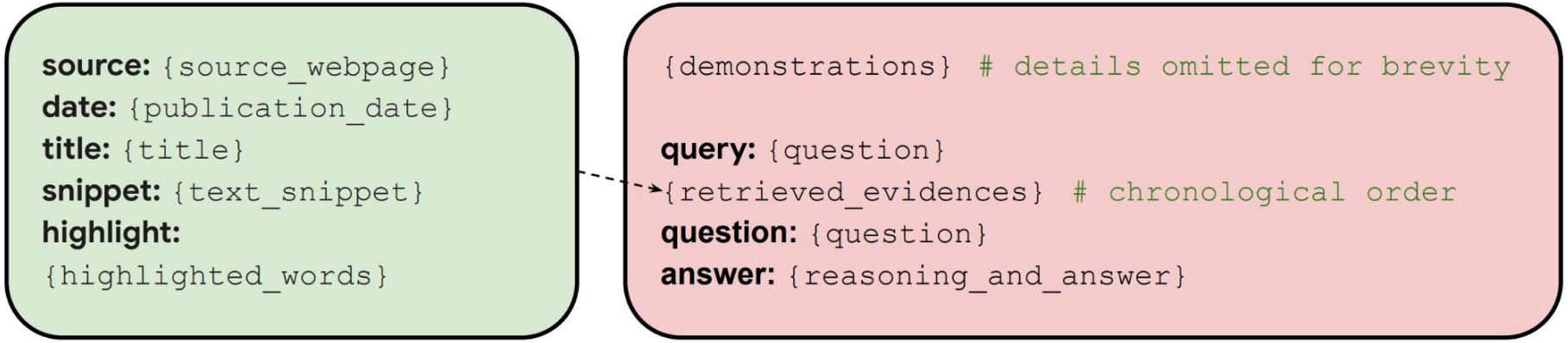
Orion Weller[*,1,2], **Michael Boratko**[1], **Iftekhar Naim**[1] **and Jinhyuk Lee**[1]
[1]Google DeepMind, [2]Johns Hopkins University

FRESHLLMS: REFRESHING LARGE LANGUAGE MODELS WITH SEARCH ENGINE AUGMENTATION

Tu Vu[1] **Mohit Iyyer**[2] **Xuezhi Wang**[1] **Noah Constant**[1] **Jerry Wei**[1]
Jason Wei[3*] **Chris Tar**[1] **Yun-Hsuan Sung**[1] **Denny Zhou**[1] **Quoc Le**[1] **Thang Luong**[1]

Google[1] University of Massachusetts Amherst[2] OpenAI[3]
freshllms@google.com

FreshPrompt



FreshPrompt uses few-shot in-context learning to teach a model to reason over retrieved evidences and figure out the right answer

Unleash the Power of Perplexity AI's Fresh Prompt Approach

Updated on Feb 28,2024

THENEWSTACK NEWSLETTER

TNS Daily Newsletter

Get our newsletter with all the most important updates about at-scale software development.

Subscribe

AI / LARGE LANGUAGE MODELS

How Perplexity's Online LLM Was Inspired by FreshLLMs Paper

We dig into the technology behind Perplexity's Copilot, which was inspired by the FreshLLMs paper that proposed search engine-augmented LLMs.

Jan 24th, 2024 4:00am by Janakiram MSV



Perplexity AI's Fresh Prompt Approach

e

Online LLMs

lels

REACT: SYNERGIZING REASONING AND ACTING IN LANGUAGE MODELS

Shunyu Yao[*,1], Jeffrey Zhao[2], Dian Yu[2], Nan Du[2], Izhak Shafran[2], Karthik Narasimhan[1], Yuan Cao[2]

[1]Department of Computer Science, Princeton University
[2]Google Research, Brain team
[1]{shunyuy,karthikn}@princeton.edu
[2]{jeffreyzhao,dianyu,dunan,izhak,yuancao}@google.com

ReAct

a few-shot prompting method that uses demonstrations of interleaved reasoning and tool use

(1) Hotspot QA

Question: Aside from the Apple Remote, what other device can control the program Apple Remote was originally designed to interact with?

(1a) Standard

Answer: iPod ✗

(1b) CoT (Reason Only)

Thought: Let's think step by step. Apple Remote was originally designed to interact with Apple TV. Apple TV can be controlled by iPhone, iPad, and iPod Touch. So the answer is iPhone, iPad, and iPod Touch.

Answer: iPhone, iPad, iPod Touch ✗

(1c) Act-Only

Act 1: Search[Apple Remote]
Obs 1: The Apple Remote is a remote control …

Act 2: Search[Front Row]
Obs 2: Could not find [Front Row]. Similar: …

Act 3: Search[Front Row (software)]
Obs 3: Front Row is a discontinued media center software …

Act 4: Finish[yes] ✗

(1d) ReAct (Reason + Act)

Thought 1: I need to search Apple Remote and find the program it was originally designed to interact with.
Act 1: Search[Apple Remote]
Obs 1: The Apple Remote is a remote control introduced in October 2005 by Apple … originally designed to control the Front Row media center program …

Thought 2: Apple Remote was originally designed to control the Front Row media center program. I need to search Front Row next and find what other device can control it.
Act 2: Search[Front Row]
Obs 2: Could not find [Front Row]. Similar: ['Front Row Seat to Earth', 'Front Row Motorsports',' Front Row (software)', …]

Thought 3: Front Row is not found. I need to search Front Row (software) .
Act 3: Search[Front Row (software)]
Obs 3: Front Row is a discontinued media center software …

Thought 4: Front Row (software) is controlled by an Apple Remote or the keyboard function keys. So the answer is keyboard function keys.
Act 4: Finish[keyboard function keys] ✓

specifies a reasoning-and-action pattern

Toolformer: Language Models Can Teach Themselves to Use Tools

Timo Schick **Jane Dwivedi-Yu** **Roberto Dessì**[†] **Roberta Raileanu**
Maria Lomeli **Luke Zettlemoyer** **Nicola Cancedda** **Thomas Scialom**

Meta AI Research [†]Universitat Pompeu Fabra

Exemplary predictions of Toolformer

The New England Journal of Medicine is a registered trademark of [QA("Who is the publisher of The New England Journal of Medicine?") → Massachusetts Medical Society] the MMS.

Out of 1400 participants, 400 (or [Calculator(400 / 1400) → 0.29] 29%) passed the test.

The name derives from "la tortuga", the Spanish word for [MT("tortuga") → turtle] turtle.

The Brown Act is California's law [WikiSearch("Brown Act") → The Ralph M. Brown Act is an act of the California State Legislature that guarantees the public's right to attend and participate in meetings of local legislative bodies.] that requires legislative bodies, like city councils, to hold their meetings open to the public.

Using in-context learning to generate API calls

Use an LLM to annotate a huge language modeling dataset with potential API calls

Your task is to add calls to a Question Answering API to a piece of text. The questions should help you get information required to complete the text. You can call the API by writing "[QA(question)]" where "question" is the question you want to ask. Here are some examples of API calls:

Input: Joe Biden was born in Scranton, Pennsylvania.

Output: Joe Biden was born in **[QA("Where was Joe Biden born?")]** Scranton, **[QA("In which state is Scranton?")]** Pennsylvania.

Input: Coca-Cola, or Coke, is a carbonated soft drink manufactured by the Coca-Cola Company.

Output: Coca-Cola, or **[QA("What other name is Coca-Cola known by?")]** Coke, is a carbonated soft drink manufactured by **[QA("Who manufactures Coca-Cola?")]** the Coca-Cola Company.

Input: x

Output:

The actual Toolformer serialization uses special tokens:

Plain text

```
Out of 1400 participants, 400 (or
<API>Calculator(400 / 1400)<ARROW>0.29<END_API>
29%) passed the test.
```

Here, `<API>`, `<ARROW>`, and `<END_API>` are readable representations of special delimiters.

Input prefix	Next-token target
`Out of 1400 participants, 400 (or`	`<API>`
`... <API>`	`Calculator`
`... Calculator(400 / 1400)`	`<ARROW>`
`... <ARROW>`	`0.29`
`... 0.29<END_API>`	`29`
`... 29`	`%`

TRAINING SEQUENCE

Out of 1400 ... (or [Calculator(400 / 1400) → 0.29] 29%) passed the test.

- ✓ Loss on ordinary text and the API call
- ⊗ No loss on the tool's returned content

A. No tool call

Out of 1400 participants, 400 (or **29%**)

B. Tool call with result

Out of 1400 participants, 400 (or [Calculator(400 / 1400) → 0.29] **29%**

C. Tool call without result

Out of 1400 participants, 400 (or [Calculator(400 / 1400) →] **29%**

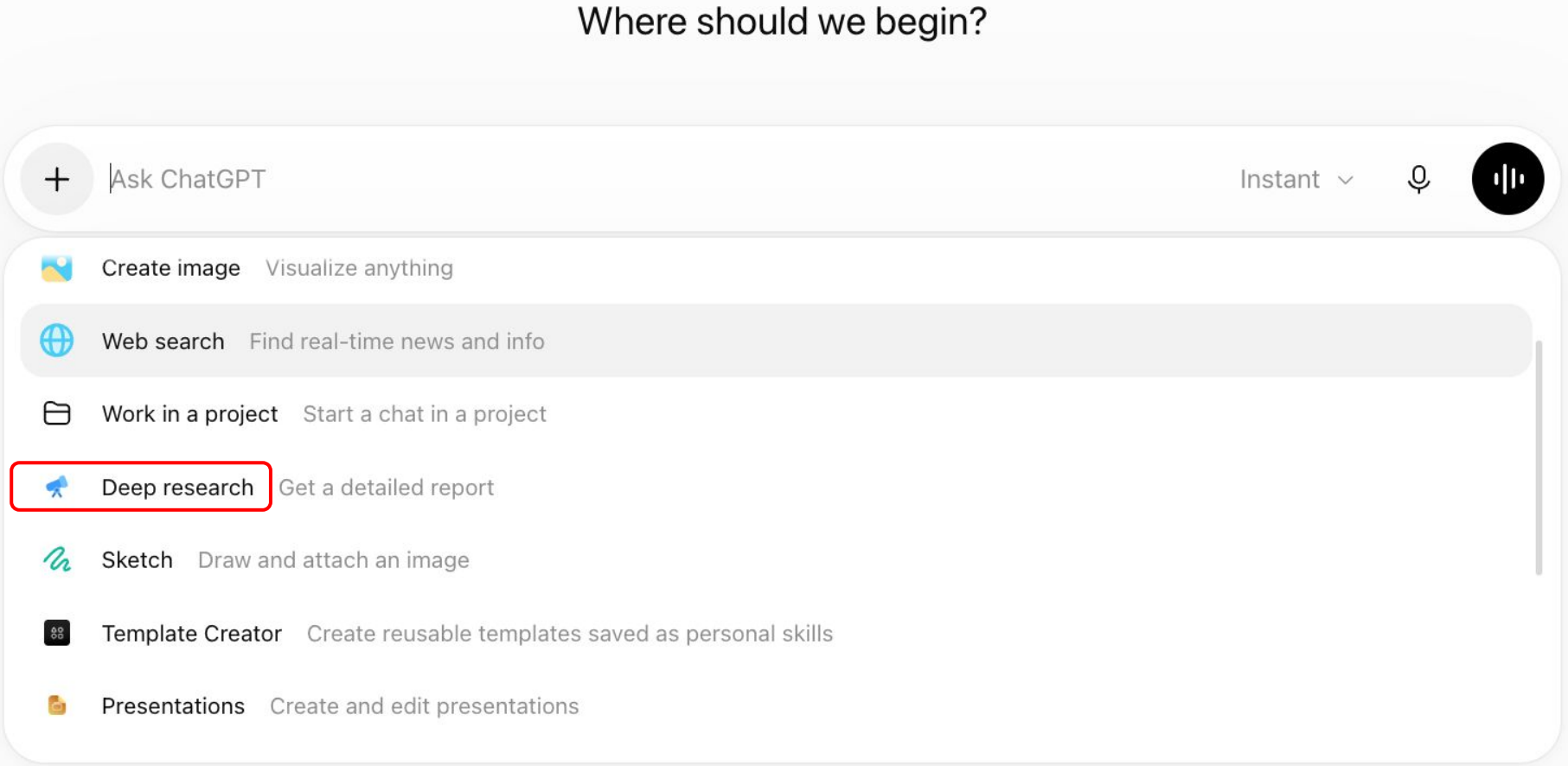
a filtering procedure to decide which API calls are useful

Toolformer (6.7B) achieves much stronger zero-shot results than OPT (66B) and GPT-3 (175B)

Model	ASDiv	SVAMP	MAWPS
GPT-J	7.5	5.2	9.9
GPT-J + CC	9.6	5.0	9.3
Toolformer (disabled)	14.8	6.3	15.0
Toolformer	**40.4**	**29.4**	**44.0**
OPT (66B)	6.0	4.9	7.9
GPT-3 (175B)	14.0	10.0	19.8

Model	WebQS	NQ	TriviaQA
GPT-J	18.5	12.8	43.9
GPT-J + CC	18.4	12.2	45.6
Toolformer (disabled)	18.9	12.6	46.7
Toolformer	**26.3**	**17.7**	**48.8**
OPT (66B)	18.6	11.4	45.7
GPT-3 (175B)	29.0	22.6	65.9

Deep Research



Deep Research (cont’d)

- https://openai.com/index/introducing-deep-research/
- An agent that uses reasoning to synthesize large amounts of online information and complete multi-step research tasks for you.

Deep Research (cont’d)

https://openai.com/index/introducing-deep-research/

Deep research is OpenAI's next agent that can do work for you independently—you give it a prompt, and ChatGPT will find, analyze, and synthesize hundreds of online sources to create a comprehensive report at the level of a research analyst. Powered by a version of the upcoming OpenAI o3 model that’s optimized for web browsing and data analysis, it leverages reasoning to search, interpret, and analyze massive amounts of text, images, and PDFs on the internet, pivoting as needed in reaction to information it encounters.

Deep Research (cont'd)

Deep research is built for people who do intensive knowledge work in areas like finance, science, policy, and engineering and need thorough, precise, and reliable research. It can be equally useful for discerning shoppers looking for hyper-personalized recommendations on purchases that typically require careful research, like cars, appliances, and furniture. Every output is fully documented, with clear citations and a summary of its thinking, making it easy to reference and verify the information. It is particularly effective at finding niche, non-intuitive information that would require browsing numerous websites. Deep research frees up valuable time by allowing you to offload and expedite complex, time-intensive web research with just one query.

Deep Research (cont'd)

Deep research independently discovers, reasons about, and consolidates insights from across the web. To accomplish this, it was trained on real-world tasks requiring browser and Python tool use, using the same reinforcement learning methods behind OpenAI o1, our first reasoning model. While o1 demonstrates impressive capabilities in coding, math, and other technical domains, many real-world challenges demand extensive context and information gathering from diverse online sources. Deep research builds on these reasoning capabilities to bridge that gap, allowing it to take on the types of problems people face in work and everyday life.

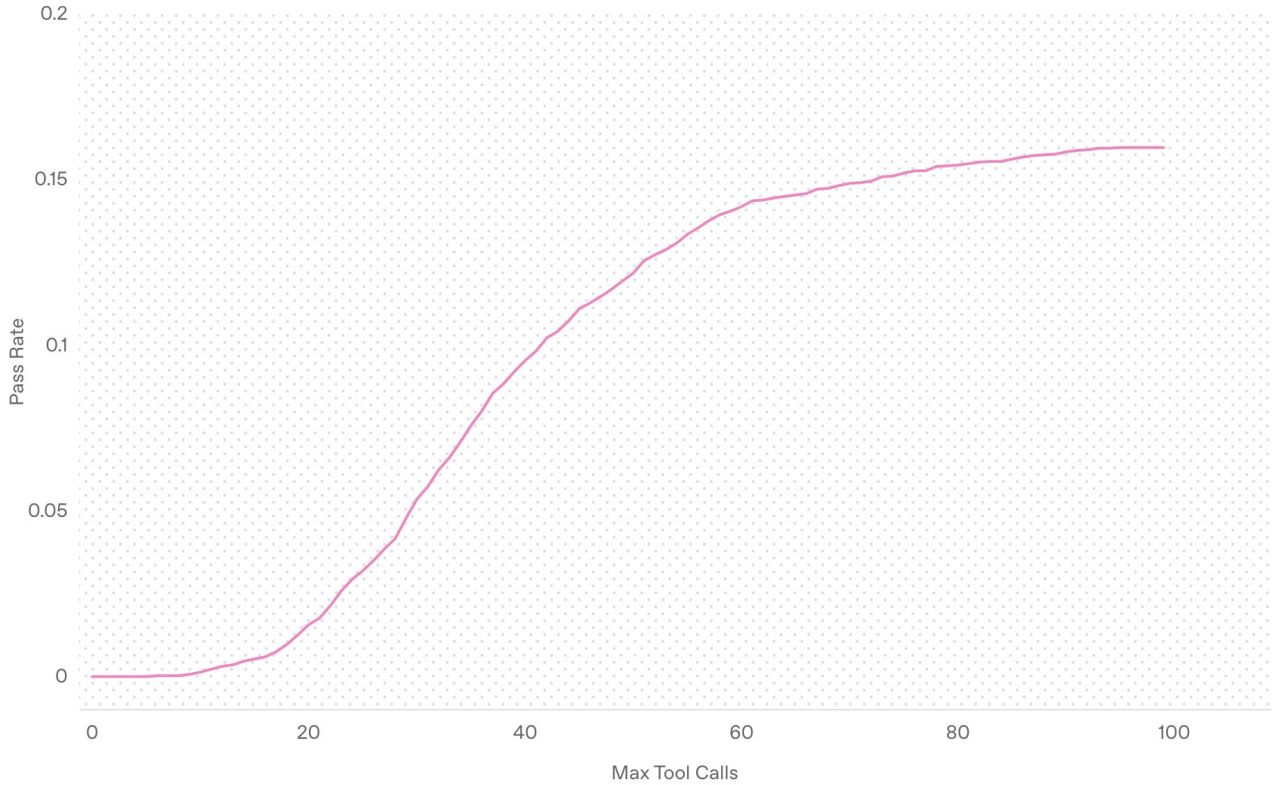
Deep Research on Humanity's Last Exam

Model	Accuracy (%)
GPT-4o	3.3
Grok-2	3.8
Claude 3.5 Sonnet	4.3
Gemini Thinking	6.2
OpenAI o1	**9.1**
DeepSeek-R1*	9.4
OpenAI o3-mini (medium)*	10.5
OpenAI o3-mini (high)*	13.0
OpenAI deep research**	**26.6**

* Model is not multi-modal, evaluated on text-only subset.

**with browsing + python tools

The more the model browses and thinks about what it is browsing, the better it does



Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning

Bowen Jin[1], **Hansi Zeng**[2], **Zhenrui Yue**[1], **Jinsung Yoon**[3], **Sercan Ö. Arık**[3], **Dong Wang**[1], **Hamed Zamani**[2], **Jiawei Han**[1]
[1] Department of Computer Science, University of Illinois at Urbana-Champaign
[2] Center for Intelligent Information Retrieval, University of Massachusetts Amherst
[3] Google Cloud AI Research
{bowenj4,zhenrui3,dwang24,hanj}@illinois.edu, {hzeng, zamani}@cs.umass.edu

Training template

Answer the given question. You must conduct reasoning inside <think> and </think> first every time you get new information. After reasoning, if you find you lack some knowledge, you can call a search engine by <search> query </search>, and it will return the top searched results between <information> and </information>. You can search as many times as you want. If you find no further external knowledge needed, you can directly provide the answer inside <answer> and </answer> without detailed illustrations. For example, <answer> xxx </answer>. Question: question.

Search-R1's algorithm

Algorithm 1 LLM Response Rollout with Multi-Turn Search Engine Calls

Require: Input query x, policy model π_θ, search engine $\mathcal{R}$, maximum action budget B.
Ensure: Final response y.

Initialize rollout sequence $y \leftarrow \emptyset$
Initialize action count $b \leftarrow 0$
while $b < B$ **do**
    Initialize current action LLM rollout sequence $y_b \leftarrow \emptyset$
    while True **do**
        Generate response token $y_t \sim \pi_\theta(\cdot \mid x, y + y_b)$
        Append y_t to rollout sequence $y_b \leftarrow y_b + y_t$
        if y_t in [</search>, </answer>, <eos>] **then** break
        end if
    end while
    $y \leftarrow y + y_b$
    if <search> </search> detected in y_b **then**
        Extract search query $q \leftarrow \text{Parse}(y_b, \texttt{<search>}, \texttt{</search>})$
        Retrieve search results $d = \mathcal{R}(q)$
        Insert d into rollout $y \leftarrow y +$ <information>d</information>
    else if <answer> </answer> detected in y_b **then**
        return final generated response y
    else
        Ask for rethink $y \leftarrow y +$ "My action is not correct. Let me rethink."
    end if
    Increment action count $b \leftarrow b + 1$
end while
return final generated response y

Search-R1's performance

Methods	General QA			Multi-Hop QA				
	NQ†	TriviaQA⋆	PopQA⋆	HotpotQA†	2wiki⋆	Musique⋆	Bamboogle⋆	Avg.
Qwen2.5-7b-Base/Instruct								
Direct Inference	0.134	0.408	0.140	0.183	0.250	0.031	0.120	0.181
CoT	0.048	0.185	0.054	0.092	0.111	0.022	0.232	0.106
IRCoT	0.224	0.478	0.301	0.133	0.149	0.072	0.224	0.239
Search-o1	0.151	0.443	0.131	0.187	0.176	0.058	0.296	0.206
RAG	0.349	0.585	0.392	0.299	0.235	0.058	0.208	0.304
SFT	0.318	0.354	0.121	0.217	0.259	0.066	0.112	0.207
R1-base	0.297	0.539	0.202	0.242	0.273	0.083	0.296	0.276
R1-instruct	0.270	0.537	0.199	0.237	0.292	0.072	0.293	0.271
Search-R1-base	**0.480**	**0.638**	**0.457**	**0.433**	0.382	**0.196**	**0.432**	**0.431**
Search-R1-instruct	0.393	0.610	0.397	0.370	**0.414**	0.146	0.368	0.385
Qwen2.5-3b-Base/Instruct								
Direct Inference	0.106	0.288	0.108	0.149	0.244	0.020	0.024	0.134
CoT	0.023	0.032	0.005	0.021	0.021	0.002	0.000	0.015
IRCoT	0.111	0.312	0.200	0.164	0.171	0.067	0.240	0.181
Search-o1	0.238	0.472	0.262	0.221	0.218	0.054	**0.320**	0.255
RAG	0.348	0.544	0.387	0.255	0.226	0.047	0.080	0.270
SFT	0.249	0.292	0.104	0.186	0.248	0.044	0.112	0.176
R1-base	0.226	0.455	0.173	0.201	0.268	0.055	0.224	0.229
R1-instruct	0.210	0.449	0.171	0.208	0.275	0.060	0.192	0.224
Search-R1-base	**0.406**	**0.587**	**0.435**	0.284	0.273	0.049	0.088	0.303
Search-R1-instruct	0.341	0.545	0.378	**0.324**	**0.319**	**0.103**	0.264	**0.325**

RAG vs. long-context LLMs

RETRIEVAL MEETS LONG CONTEXT LARGE LANGUAGE MODELS

Peng Xu[†], Wei Ping[†], Xianchao Wu, Lawrence McAfee
Chen Zhu, Zihan Liu, Sandeep Subramanian, Evelina Bakhturina
Mohammad Shoeybi, Bryan Catanzaro
NVIDIA
[†]{pengx, wping}@nvidia.com

ABSTRACT

Extending the context window of large language models (LLMs) is getting popular recently, while the solution of augmenting LLMs with retrieval has existed for years. The natural questions are: *i) Retrieval-augmentation versus long context window, which one is better for downstream tasks? ii) Can both methods be combined to get the best of both worlds?* In this work, we answer these questions by studying both solutions using two state-of-the-art pretrained LLMs, i.e., a proprietary 43B GPT and Llama2-70B. Perhaps surprisingly, we find that LLM with 4K context window using simple retrieval-augmentation at generation can achieve comparable performance to finetuned LLM with 16K context window via *positional interpolation* on long context tasks, while taking much less computation. More importantly, we demonstrate that retrieval can significantly improve the performance of LLMs regardless of their extended context window sizes. Our best model, retrieval-augmented Llama2-70B with 32K context window, outperforms GPT-3.5-turbo-16k and Davinci003 in terms of average score on nine long context tasks including question answering, query-based summarization, and in-context few-shot learning tasks. It also outperforms its non-retrieval Llama2-70B-32k baseline by a margin, while being much faster at generation. Our study provides general insights on the choice of retrieval-augmentation versus long context extension of LLM for practitioners.

Can Long-Context Language Models Subsume Retrieval, RAG, SQL, and More?

Jinhyuk Lee* **Anthony Chen*** **Zhuyun Dai***
Dheeru Dua **Devendra Singh Sachan** **Michael Boratko** **Yi Luan**
Sébastien M. R. Arnold **Vincent Perot** **Siddharth Dalmia** **Hexiang Hu**
Xudong Lin **Panupong Pasupat** **Aida Amini** **Jeremy R. Cole**
Sebastian Riedel **Iftekhar Naim** **Ming-Wei Chang** **Kelvin Guu**

Google DeepMind

Abstract

Long-context language models (LCLMs) have the potential to revolutionize our approach to tasks traditionally reliant on external tools like retrieval systems or databases. Leveraging LCLMs' ability to natively ingest and process entire corpora of information offers numerous advantages. It enhances user-friendliness by eliminating the need for specialized knowledge of tools, provides robust end-to-end modeling that minimizes cascading errors in complex pipelines, and allows for the application of sophisticated prompting techniques across the entire system. To assess this paradigm shift, we introduce LOFT, a benchmark of real-world tasks requiring context up to millions of tokens designed to evaluate LCLMs' performance on in-context retrieval and reasoning. Our findings reveal LCLMs' surprising ability to rival state-of-the-art retrieval and RAG systems, despite never having been explicitly trained for these tasks. However, LCLMs still face challenges in areas like compositional reasoning that are required in SQL-like tasks. Notably, prompting strategies significantly influence performance, emphasizing the need for continued research as context lengths grow. Overall, LOFT provides a rigorous testing ground for LCLMs, showcasing their potential to supplant existing paradigms and tackle novel tasks as model capabilities scale.[1]

Retrieval Augmented Generation or Long-Context LLMs? A Comprehensive Study and Hybrid Approach

Zhuowan Li[1] **Cheng Li**[1] **Mingyang Zhang**[1]
Qiaozhu Mei[2*] **Michael Bendersky**[1]
[1] Google DeepMind [2] University of Michigan
[1] {zhuowan,chgli,mingyang,bemike}@google.com [2] qmei@umich.edu

Abstract

Retrieval Augmented Generation (RAG) has been a powerful tool for *Large Language Models (LLMs)* to efficiently process overly lengthy contexts. However, recent LLMs like Gemini-1.5 and GPT-4 show exceptional capabilities to understand long contexts directly. We conduct a comprehensive comparison between RAG and long-context *(LC)* LLMs, aiming to leverage the strengths of both. We benchmark RAG and LC across various public datasets using three latest LLMs. Results reveal that when resourced sufficiently, LC consistently outperforms RAG in terms of average performance. However, RAG's significantly lower cost remains a distinct advantage. Based on this observation, we propose SELF-ROUTE, a simple yet effective method that routes queries to RAG or LC based on model self-reflection. SELF-ROUTE significantly reduces the computation cost while maintaining a comparable performance to LC. Our findings provide a guideline for long-context applications of LLMs using RAG and LC.

You don't need long context

Peter Belcak, Saurav Muralidharan, Jindong Jiang, Yonggan Fu, Wonmin Byeon, Hanrong Ye, Ryan Angilly, Ali Taghibakhshi, Shih-Yang Liu, Siru Ouyang, Greg Heinrich, Pavlo Molchanov

Abstract. Later tokens are costlier to generate than earlier tokens. On frontier open weight models, the disparity in cost may rise to as much as 7x. Through frequent compaction, we show across a broad range of long-horizon and long-input tasks that frontier language models sacrifice little to no task accuracy when maintaining short contexts. We further demonstrate that LLM performance does not collapse on long-horizon software engineering tasks even when taking at most one action per context, and based on that we outline the concept of semantic state space models (SSSMs). Our findings question the need for long-context language models.

Links: Lab | Project Moonshot: Local Frontier Intelligence

3 Nov 2026

1. Introduction

Not all tokens are generated equal. Token generation latency and memory footprint both grow with the number of tokens in a language model's context. This is due to the inherent linear[1] relationship between the computational cost of generating a token and its depth within a context [25]. Figure 1 shows the cloud token generation cost as a function of context window size for a selection of popular open-weight models. Observe

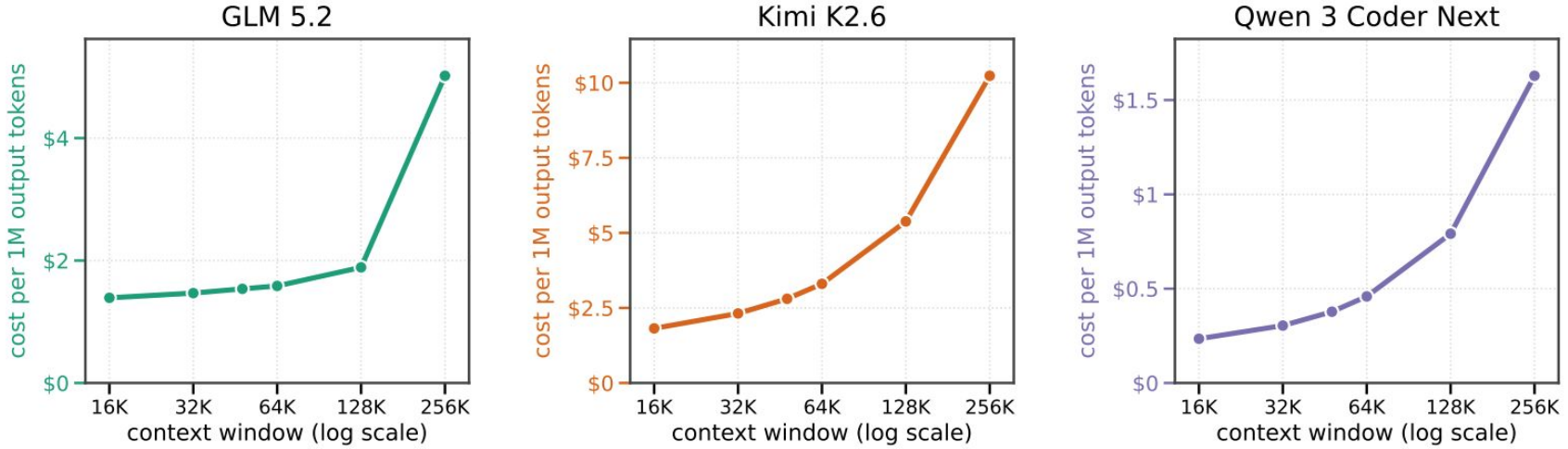


Figure 1 | **The cost of long context.** Dollar cost of generating 1M output tokens on an existing input KV cache as a function of the context window size, obtained by converting the measured peak decode throughput of each deployment at on-demand H100 prices ($3.85 per GPU-hour, Nebius, Sept. 2026). Input request lengths are drawn uniformly at random. GLM 5.2 and Kimi K2.6 are served with INT4 weights and BF16 activations, Qwen 3 Coder Next is served in BF16. Serving at a context window size of 256K rather than 16K costs 3.6× more per generated token for GLM, 5.6× more for Kimi, and 7× more for Qwen.

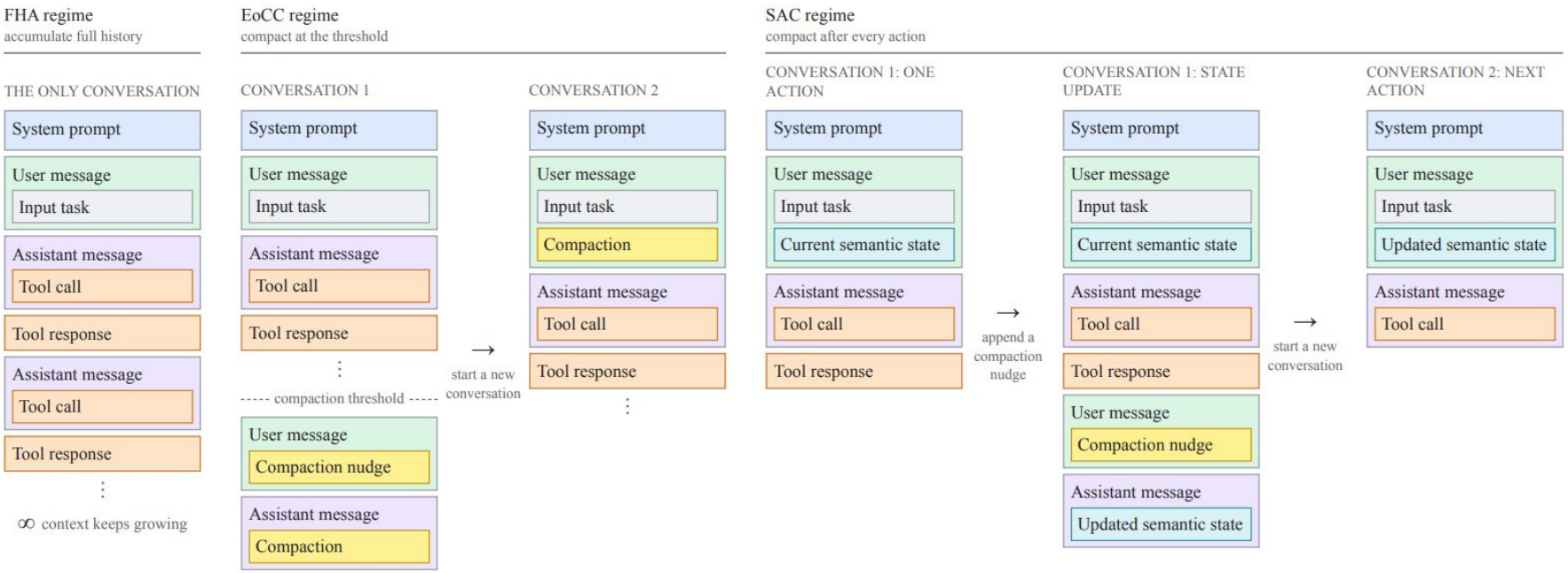
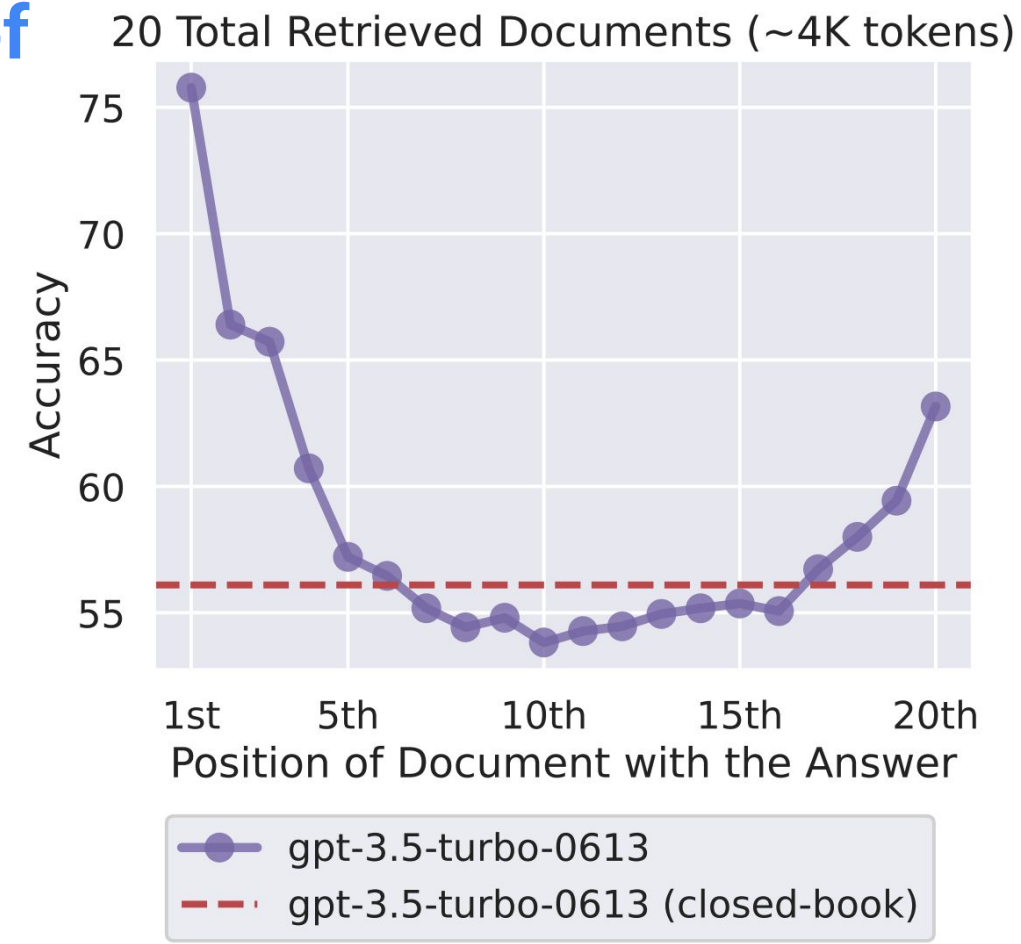


Figure 3 | **Context-management regimes.** The three context-management regimes considered in this work: Full-History Accumulation (FHA), End-of-Conversation Compaction (EoCC), and Single-Action Compaction (SAC).

Lost in the Middle: How Language Models Use Long Contexts

Nelson F. Liu[1*] **Kevin Lin**[2] **John Hewitt**[1] **Ashwin Paranjape**[3]
Michele Bevilacqua[3] **Fabio Petroni**[3] **Percy Liang**[1]
[1]Stanford University [2]University of California, Berkeley [3]Samaya AI
nfliu@cs.stanford.edu

Changing the location of relevant information results in a U-shaped performance curve



Context rot: How increasing input tokens impacts model performance

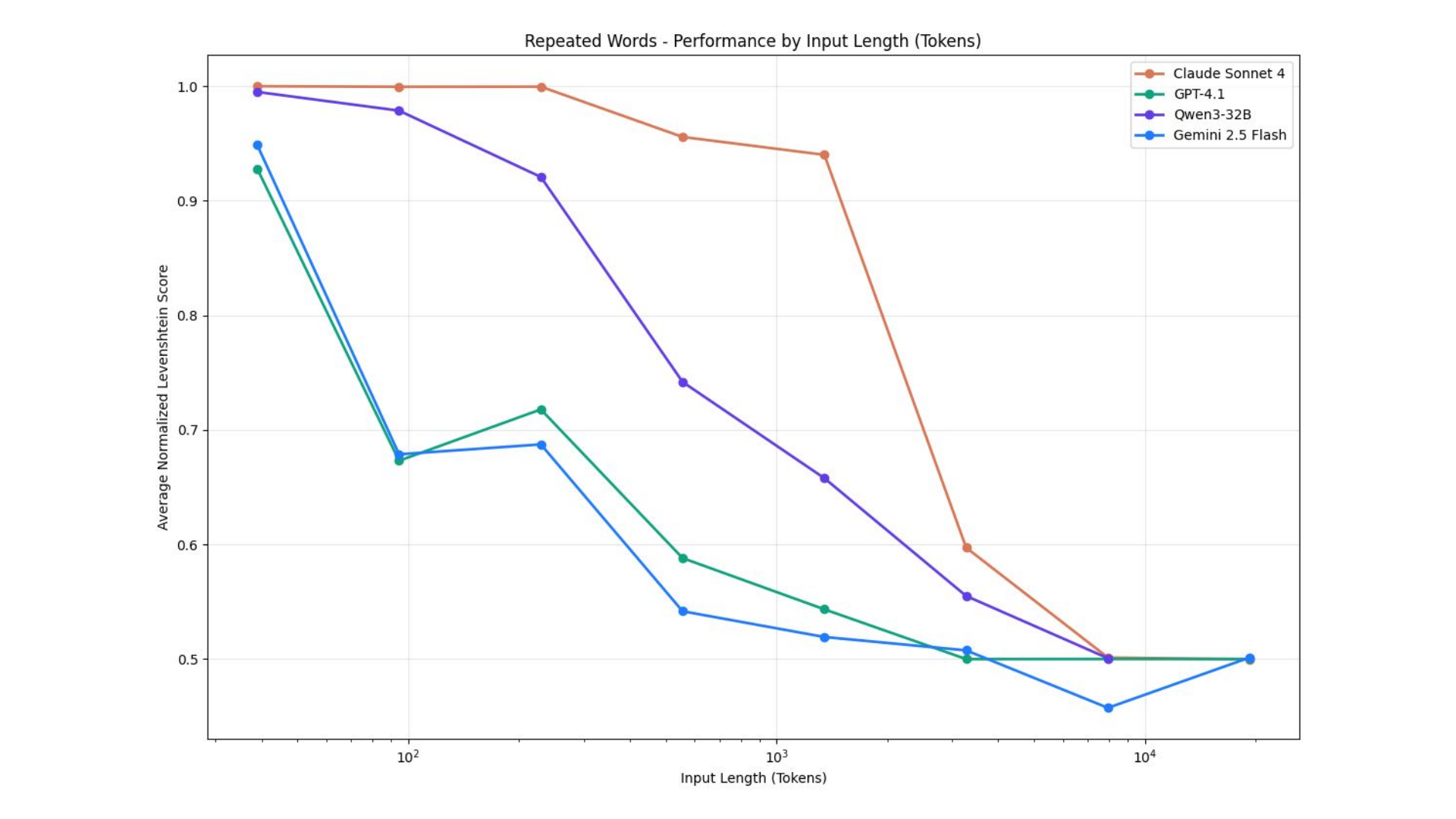
- https://research.trychroma.com/context-rot

We design a controlled task in which the model must replicate a sequence of repeated words, with a single unique word inserted at a specific position. The prompt explicitly instructs the model to reproduce the input text exactly.

One example prompt is:

```
Simply replicate the following text, output the exact same text: apple apple apple apple apples apple apple apple apple apple apple apple apple apple apple apple apple apple apple apple apple apple apple apple apple apple
```

Repeated Words - Sample Prompt Containing 'apple' as the repeated word, and 'apples' as the unique word


Repeated Words - Performance by Input Length (Tokens)
Claude Sonnet 4
GPT-4.1
Qwen3-32B
Gemini 2.5 Flash
Average Normalized Levenshtein Score
1.0
0.9
0.8
0.7
0.6
0.5
10^2
10^3
10^4
Input Length (Tokens)

v3 [cs.AI] 11 May 2026

Recursive Language Models

Alex L. Zhang
MIT CSAIL
altzhang@mit.edu

Tim Kraska
MIT CSAIL
kraska@mit.edu

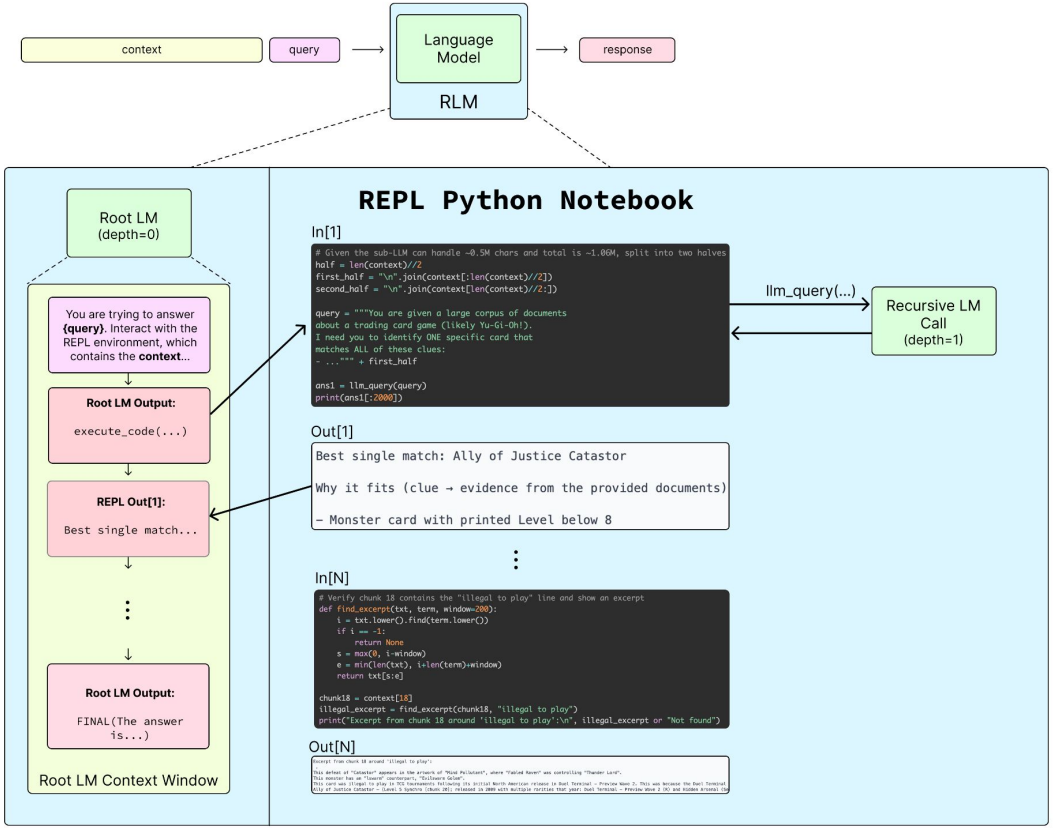
Omar Khattab
MIT CSAIL
okhattab@mit.edu

Abstract

We study allowing large language models (LLMs) to process arbitrarily long prompts through the lens of inference-time scaling. We propose **Recursive Language Models** (**RLM**s), a general inference paradigm that treats long prompts as part of an external *environment* and allows the LLM to *programmatically* examine, decompose, and *recursively call itself over* snippets of the prompt. We find that RLMs can successfully process inputs more than an order of magnitude beyond model context window limits and, even for shorter prompts, dramatically outperform the quality of vanilla frontier LLMs and common long-context and coding scaffolds (e.g., on GPT-5 by a median across the evaluated benchmarks of 26% against compaction, 130% against CodeAct with sub-calls, and 13% against Claude Code) across four diverse long-context tasks while having comparable cost. At a small scale, we post-train the first model around the RLM. Our model, **RLM-Qwen3-8B**, outperforms the underlying Qwen3-8B model by a median of 28% and even approaches the quality of vanilla GPT-5 on three long-context tasks. Code is available at `https://github.com/alexzhang13/rlm`.

Recursive Language Models

RLM with a REPL environment:



https://alexzhang13.github.io/blog/2025/rlm/

Recursive Language Models (cont'd)

Input: a long report of 10,000 words about a company's 5-year strategy
Query: What risks did the report identify in year 3?

AI: High-level scan / peek at the report's table of contents

```
# find the header line number
hdr=$(grep -inE '^(table of contents|contents|toc)\s*$' doc.txt | head -1 | cut -d: -f1)
# print the next 100 lines that look like TOC entries
tail -n +"$hdr" doc.txt | head -n 100 | grep -nE '(^\s*\d+(\.\d+)*\s+.+)|(^\s*.+\s+\.{2,}\s*\d+\s*$)'
```

Select sub-chunks
Based on that peek, it might decide: "Chapter 4 (pages 40–60) and Chapter 6 (pages 90–110) are likely to mention Year 3 risks."
Recursive call on chunks It then issues sub-queries:
"From pages 40–60, what risks are mentioned for year 3?"
"From pages 90–110, what risks are mentioned for year 3?"
Aggregate / refine
"In year 3 the report warned of supply chain disruption, regulatory changes, and declining demand in Asia."

5v1 [cs.AI] 29 Sep 2026

Context Language Models

Rulin Shao[1,2], **Shannon Zejiang Shen**[3], **Junjie Oscar Yin**[1,2], **Yuetai Li**[1], **Minheng Wang**[1], **Hamish Ivison**[1], **Radha Poovendran**[1], **Nathan Lambert**[4], **Teng Xiao**[1], **Mike Lewis**[2], **Wen-tau Yih**[2], **Luke Zettlemoyer**[1,2], **Pang Wei Koh**[1]

[1]University of Washington, [2]Meta Superintelligence Labs, [3]MIT, [4]Trillium Labs

We introduce **Context Language Models (CLMs)**, language models that natively manage their own context. We implement this by treating the **context as a file** and allowing the model to make unrestricted updates to this file. This allows the model to learn what is most important to maintain in context, and naturally extends to multi-agent systems where multiple agent contexts coexist as files. Building CLMs **zero-shot** with existing models outperforms SOTA context-management strategies across a variety of tasks: 11.4% higher accuracy with 21.5% fewer FLOPs on BrowseComp-Plus, 5% higher scores with 59% fewer FLOPs on 12-hour EdgeBench, and 65% greater improvement with the same compute on a 24-hour multi-repository agent-swarm task. Moreover, by shifting context management from external harness control to intrinsic model behavior, CLMs naturally enable both **in-context and parametric learning** of context-management strategies. We show that CLMs can be steered with natural-language instructions evolved through a standard skill-optimization loop, improving held-out accuracy by up to 35.9 points on a context-management task while reducing compute. We also introduce an online reinforcement learning method for CLMs, improving Qwen3.5-9B performance on BrowseComp-Plus by 47.6% while using 12% fewer FLOPs. Finally, we co-design **Suffix Cache Reuse** for CLM serving, further reducing server-side compute by 35% relative to standard SGLang at matched performance.

Date: September 30, 2026
Correspondence: Rulin Shao at rulin@cs.washington.edu
Code: https://github.com/facebookresearch/context-language-models

Meta

Context Language Models



(A) QUALITATIVE EXAMPLES

Writes loops to prune results

```
for t in turns:
  if irrelevant(t):
    t = "No relevant results."
```

Defines and reuses functions

```
def process_turns(s): …
```

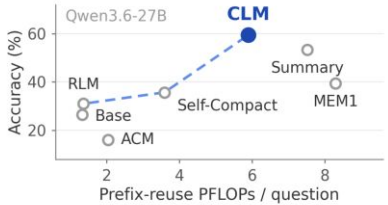
step 109 `process_turns(s)`
step 142 `process_turns(s)`

Tracks subagents in context

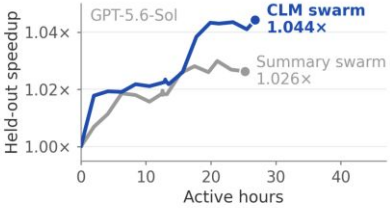
```
## STATE
BEST SO FAR 0.9992
AGENTS 21 · 5 running
```

(B) ZERO-SHOT APPLICATION

BrowseComp-Plus, 32K peak ctx

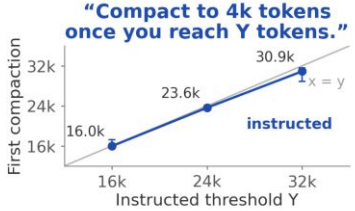


SoftwareWorld, 24hr agent swarm

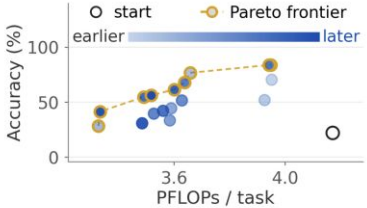


(C) LEARN IN CONTEXT

Steer in context



Evolve in context



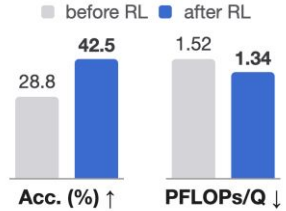
(D) LEARN IN WEIGHTS

Advantage for stepwise GRPO

$$A_i = A_i^{\text{out}} + w_{\text{eff}} \times A_i^{\text{eff}}$$

task advantage — success-gated efficiency advantage

BrowseComp-Plus, Qwen3.5-9B



Model context protocol (MCP)

- https://www.deeplearning.ai/short-courses/mcp-build-rich-context-ai-apps-with-anthropic/

Demo

- https://learn.deeplearning.ai/courses/mcp-build-rich-context-ai-apps-with-anthropic/lesson/ccsd0/why-mcp

What is the Model Context Protocol (MCP)

MCP is an open protocol that standardizes how your **LLM applications** connect to and work with your **tools & data sources**.

REST APIs

Standardize how **web applications** interact with the **backend**

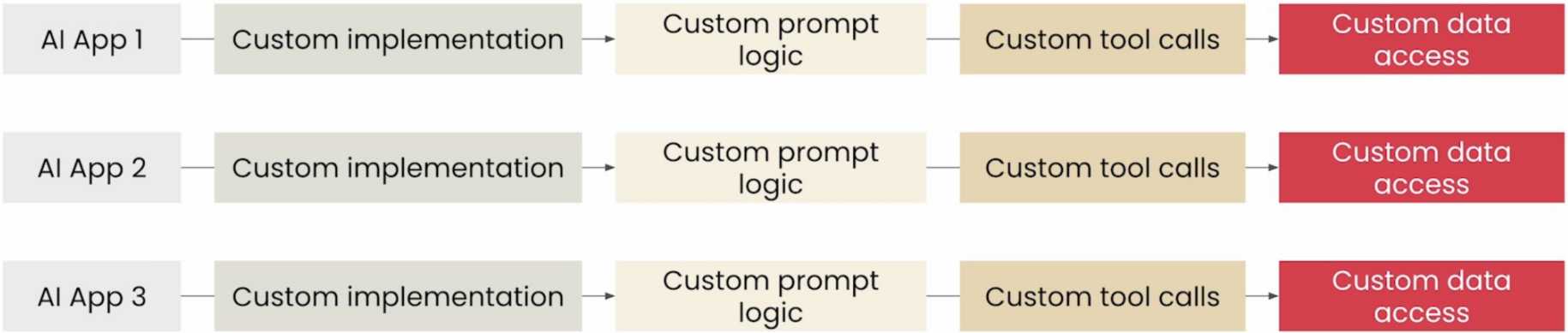
LSP

Standardizes how **IDEs** interact with **language-specific tools**

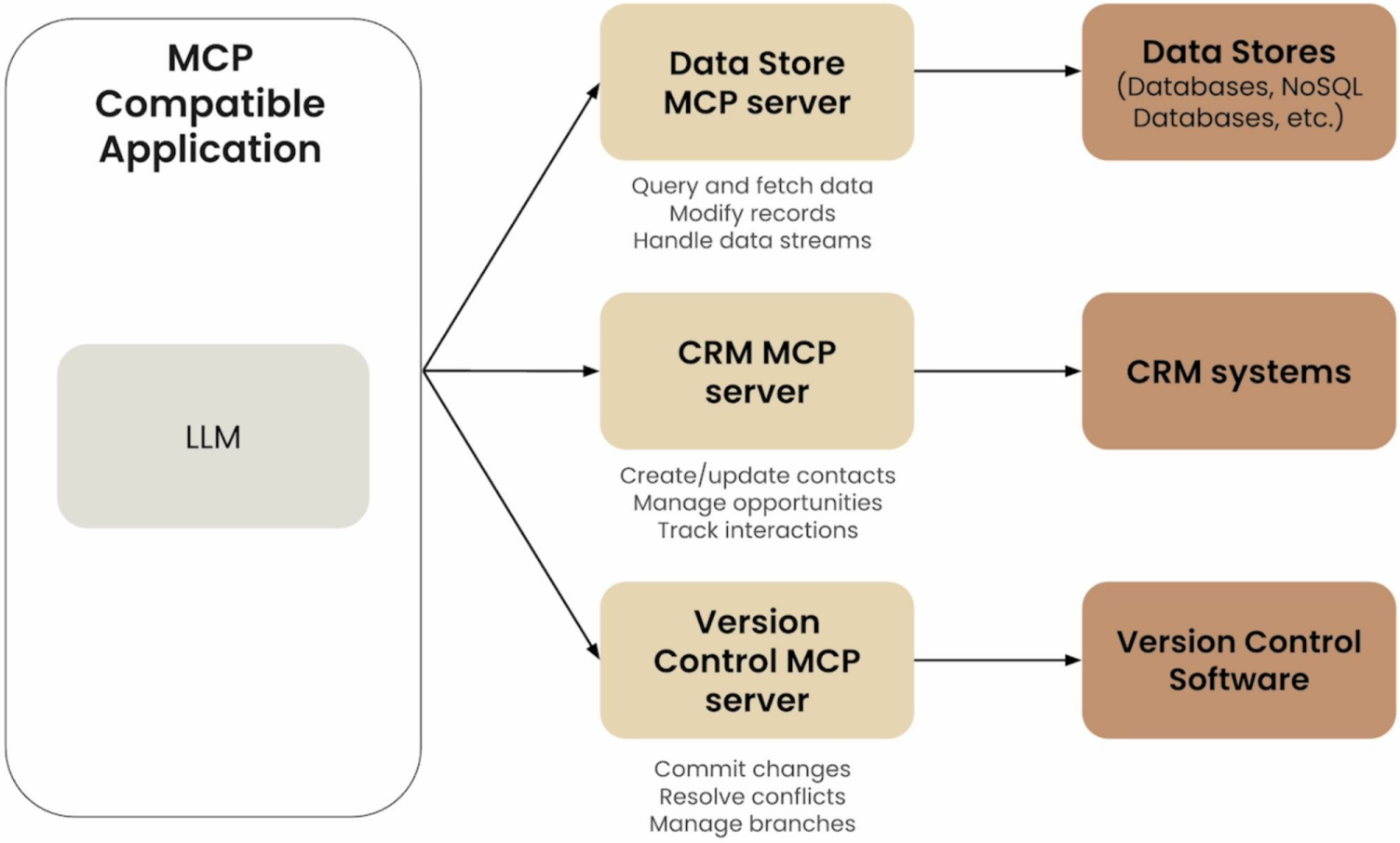
MCP

Standardizes how **AI applications** interact with **external systems**

Without MCP: Fragmented AI Development



With MCP: Standardized AI Development

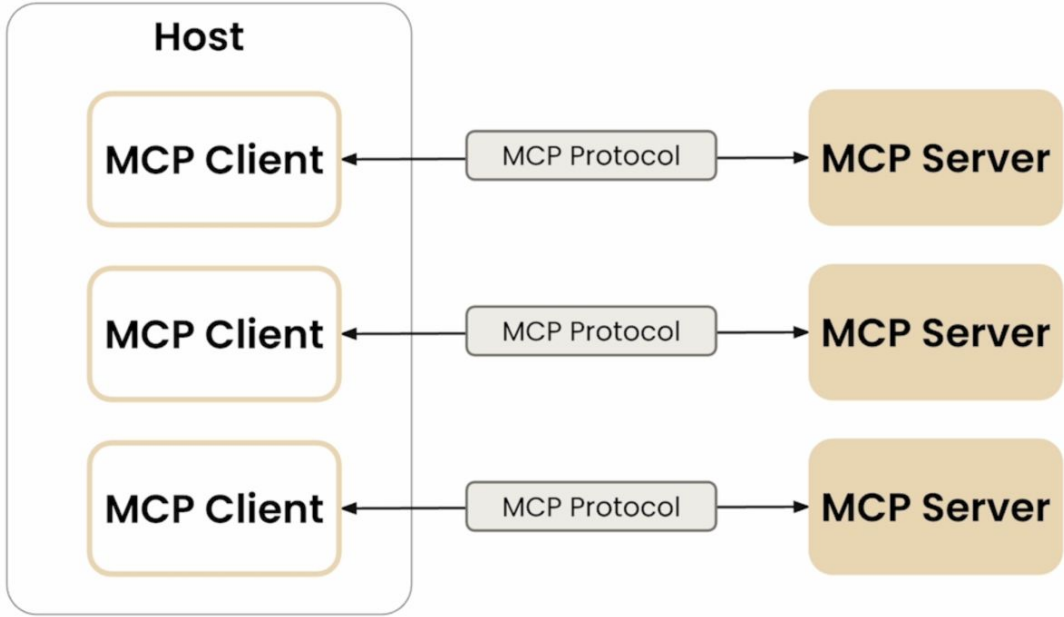


Client-Server Architecture

Host are LLM applications that want to access data through MCP (ex: Claude Desktop, IDEs, AI agents).

MCP Servers are lightweight programs that each expose specific capabilities through MCP.

MCP Clients maintain 1:1 connections with servers, inside the host application.



README | Code of conduct | MIT license | Security

Model Context Protocol servers

This repository is a collection of *reference implementations* for the Model Context Protocol (MCP), as well as references to community built servers and additional resources.

The servers in this repository showcase the versatility and extensibility of MCP, demonstrating how it can be used to give Large Language Models (LLMs) secure, controlled access to tools and data sources. Each MCP server is implemented with either the Typescript MCP SDK or Python MCP SDK.

> Note: Lists in this README are maintained in alphabetical order to minimize merge conflicts when adding new items.

Reference Servers

These servers aim to demonstrate MCP features and the TypeScript and Python SDKs.

- **AWS KB Retrieval** - Retrieval from AWS Knowledge Base using Bedrock Agent Runtime
- **Brave Search** - Web and local search using Brave's Search API
- **EverArt** - AI image generation using various models
- **Everything** - Reference / test server with prompts, resources, and tools
- **Fetch** - Web content fetching and conversion for efficient LLM usage
- **Filesystem** - Secure file operations with configurable access controls
- **Git** - Tools to read, search, and manipulate Git repositories
- **GitHub** - Repository management, file operations, and GitHub API integration
- **GitLab** - GitLab API, enabling project management
- **Google Drive** - File access and search capabilities for Google Drive
- **Google Maps** - Location services, directions, and place details
- **Memory** - Knowledge graph-based persistent memory system
- **PostgreSQL** - Read-only database access with schema inspection
- **Puppeteer** - Browser automation and web scraping
- **Redis** - Interact with Redis key-value stores
- **Sentry** - Retrieving and analyzing issues from Sentry.io
- **Sequential Thinking** - Dynamic and reflective problem-solving through thought sequences
- **Slack** - Channel management and messaging capabilities
- **Sqlite** - Database interaction and business intelligence capabilities
- **Time** - Time and timezone conversion capabilities

Third-Party Servers

Official Integrations

Agentic AI

What is the one most important AI technology to pay attention to?



I would say Agentic AI

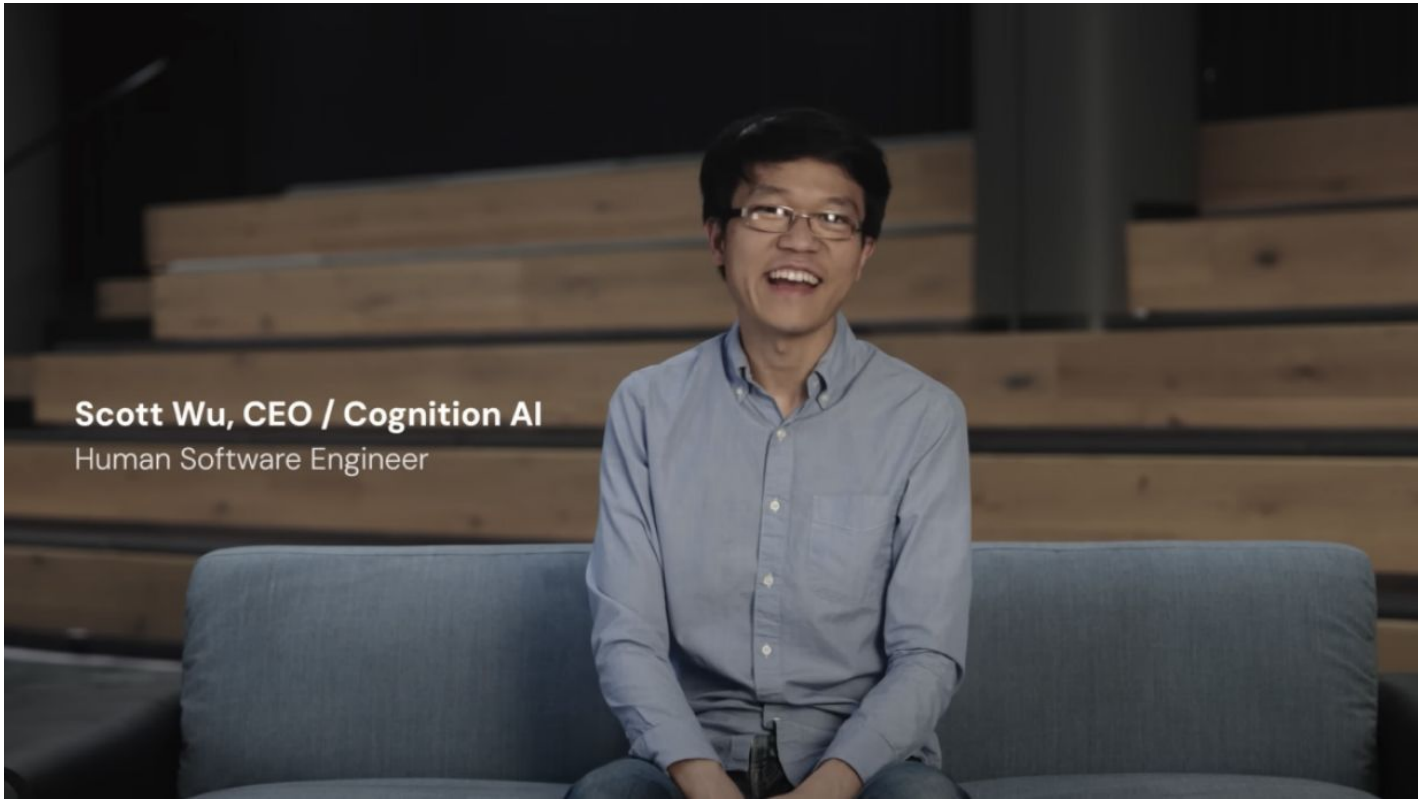
Andrew Ng



Andrew Ng

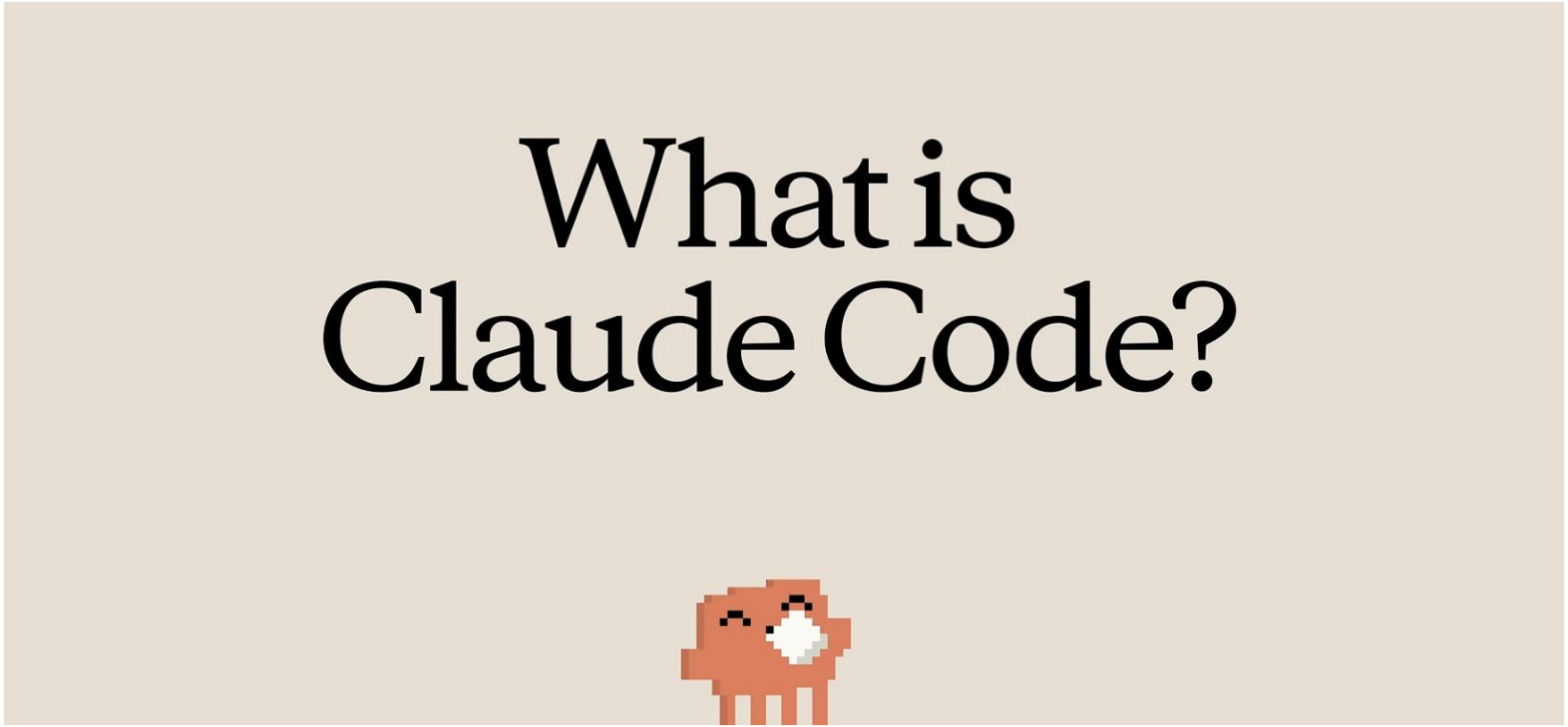
I think AI agent workflows will drive massive AI progress this year — perhaps even more than the next generation of foundation models. This is an important trend, and I urge everyone who works in AI to pay attention to it.

Devin: the first AI software engineer



https://www.youtube.com/watch?v=fjHtjT7GO1c

Claude Code

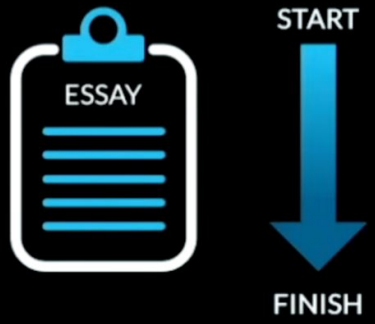


https://www.youtube.com/watch?v=fl1DSmwQKKY

AGENTIC AI

NON-AGENTIC WORKFLOW (ZERO-SHOT)

Please type out an essay on topic X from start to finish in one go, without using backspace.



AGENTIC WORKFLOW

Write an essay outline on topic X

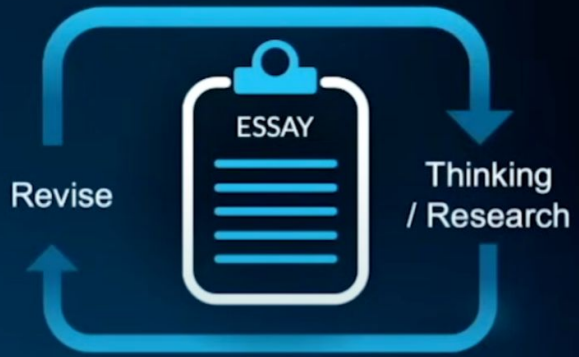
Do you need any web research?

Write a first draft.

Consider what parts need revision or more research.

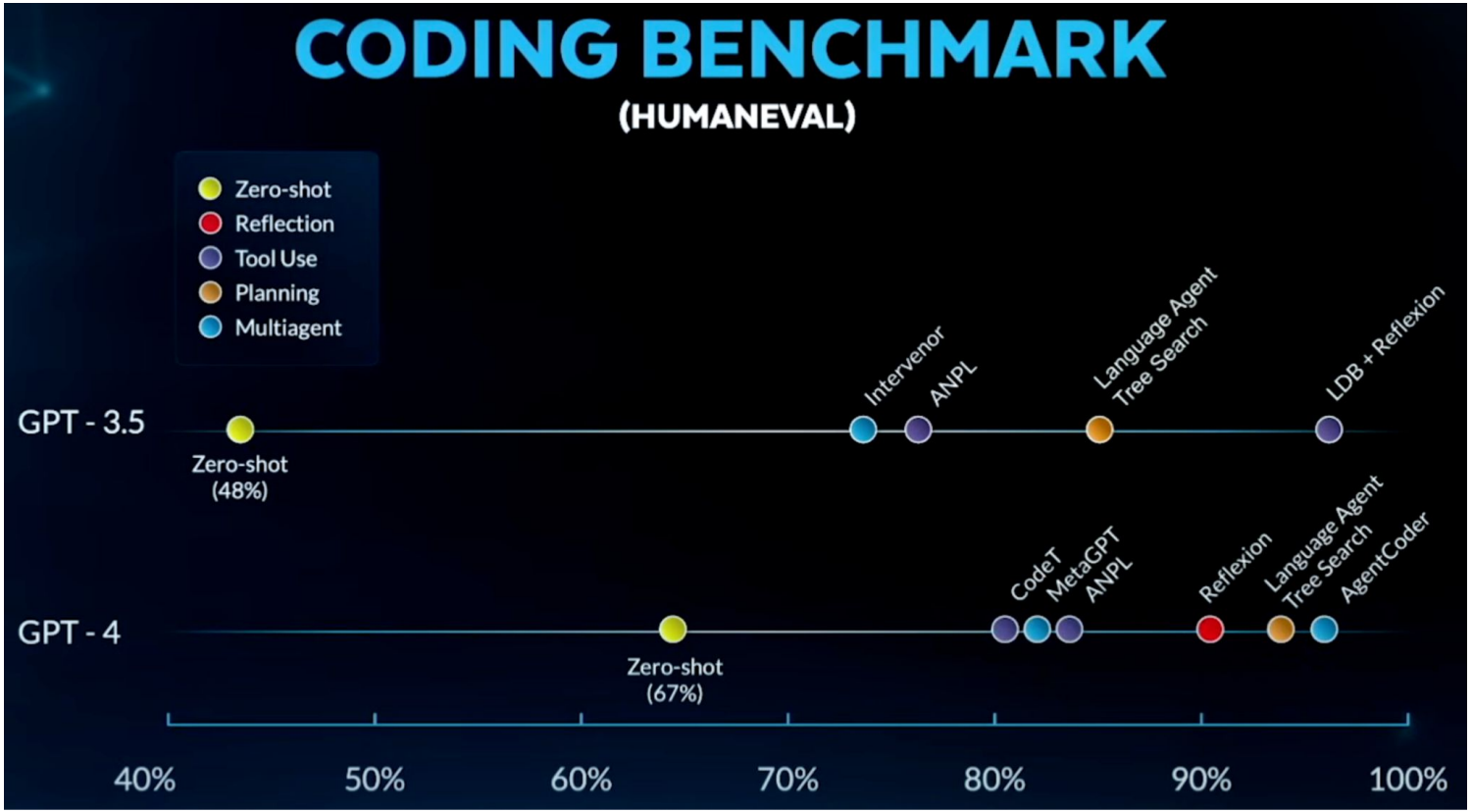
Revise your draft.

...



"Explores The Rise Of AI Agents And Agentic Reasoning" by Andrew Ng

Performance using zero-shot and agent workflows



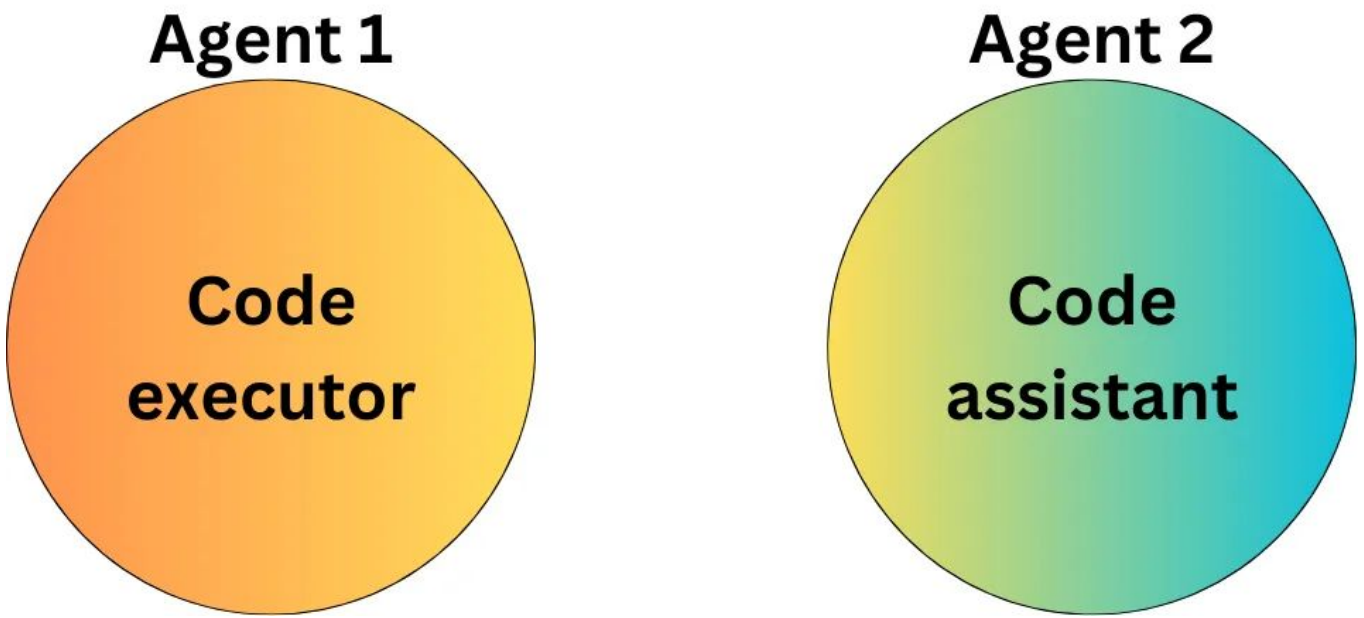
"Explores The Rise Of AI Agents And Agentic Reasoning" by Andrew Ng

Design patterns for building agents

- **Reflection:** The LLM examines its own work to come up with ways to improve it.
- **Tool Use:** The LLM is given tools such as web search, code execution, or any other function to help it gather information, take action, or process data.
- **Planning:** The LLM comes up with, and executes, a multistep plan to achieve a goal (for example, writing an outline for an essay, then doing online research, then writing a draft, and so on).
- **Multi-agent collaboration:** More than one AI agent work together, splitting up tasks and discussing and debating ideas, to come up with better solutions than a single agent would.

https://www.deeplearning.ai/the-batch/how-agents-can-improve-llm-performance/

2-agents conversation/collaboration



"The Different Agentic Patterns" by Damien Benveniste

System prompt for the code assistant

You are a helpful AI assistant.

Solve tasks using your coding and language skills.

In the following cases, suggest python code (in a python coding block) or shell script (in a sh coding block) for the user to execute.

1. When you need to collect info, use the code to output the info you need, for example, browse or search the web, download/read a file, print the content of a webpage or a file, get the current date/time, check the operating system. After sufficient info is printed and the task is ready to be solved based on your language skill, you can solve the task by yourself.

2. When you need to perform some task with code, use the code to perform the task and output the result. Finish the task smartly.

Solve the task step by step if you need to. If a plan is not provided, explain your plan first. Be clear which step uses code, and which step uses your language skill.

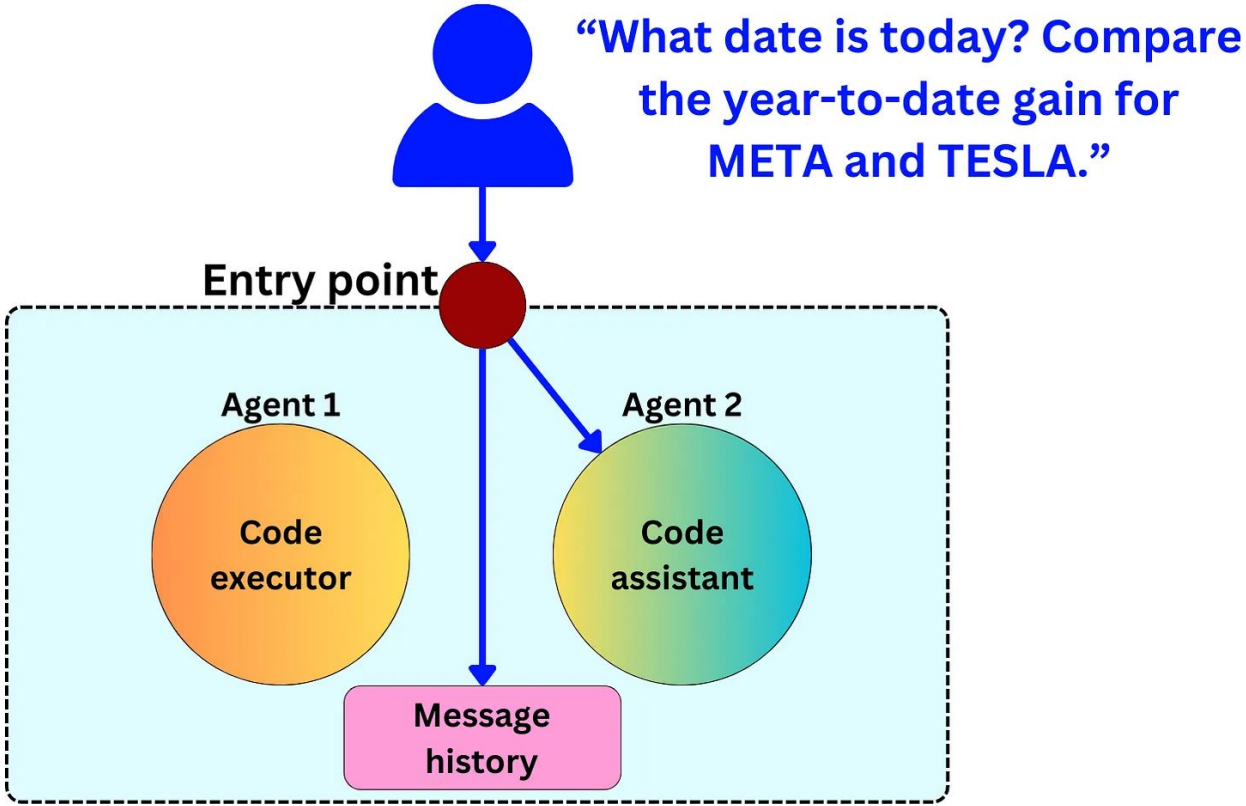
...

When you find an answer, verify the answer carefully. Include verifiable evidence in your response if possible.

Reply "TERMINATE" in the end when everything is done.

"The Different Agentic Patterns" by Damien Benveniste

2-agents conversation/collaboration (cont'd)



"The Different Agentic Patterns" by Damien Benveniste

2-agents conversation/collaboration (cont'd)

assistant (to executor)

To determine today's date and compare the year-to-date gain for META (Meta Platforms, Inc.) and TESLA (Tesla, Inc.), I will follow these steps:

1. Get today's date.

2. Fetch the current stock prices for META and TESLA.

3. Fetch the stock prices for META and TESLA at the beginning of the year.

4. Calculate the year-to-date gain for both stocks.

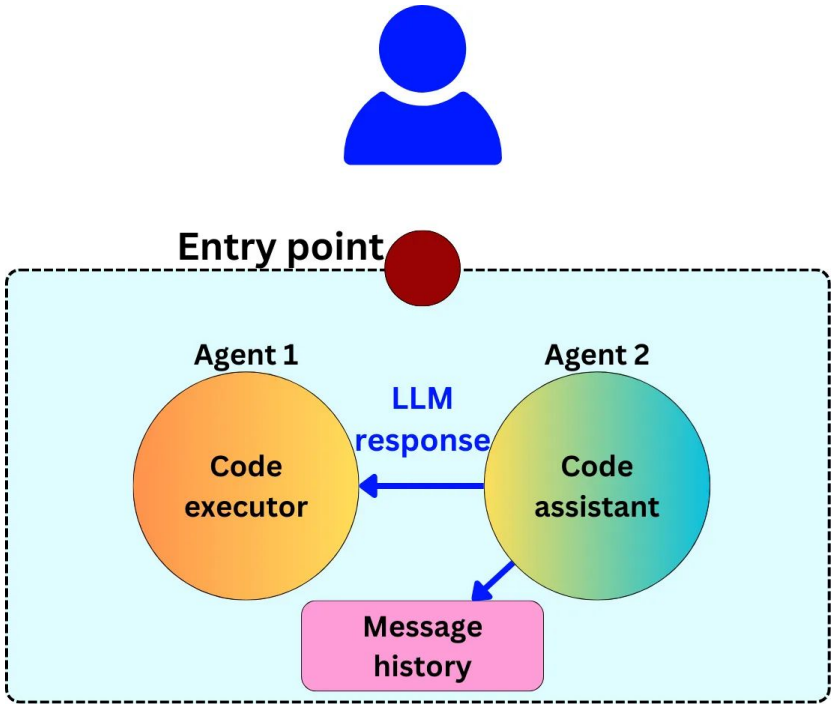
Let's start by getting today's date. I will execute the code to print the current date.

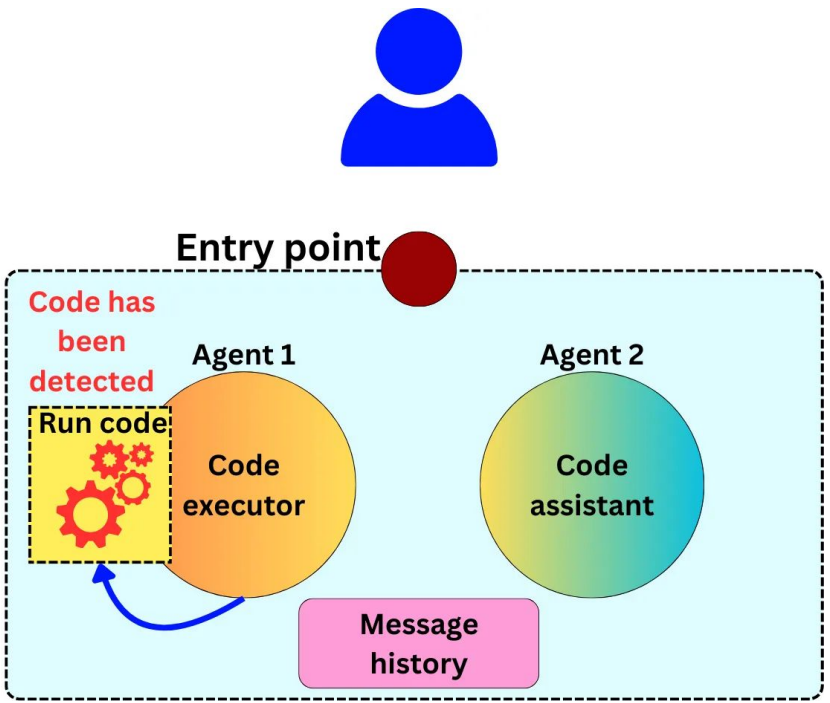
```python
from datetime import datetime

# Get today's date
today = datetime.now().date()
print(today)
```

"The Different Agentic Patterns" by Damien Benveniste

2-agents conversation/collaboration (cont'd)





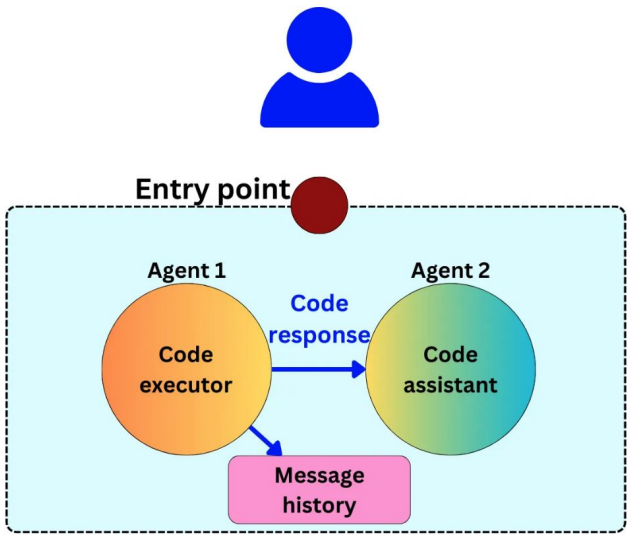
"The Different Agentic Patterns" by Damien Benveniste

2-agents conversation/collaboration (cont'd)

```
user_proxy (to assistant):
```

exitcode: 0 (execution succeeded)

Code output: 2024-09-17



"The Different Agentic Patterns" by Damien Benveniste

2-agents conversation/collaboration (cont'd)

assistant (to executor):

Today's date is September 17, 2024.

Next, I will fetch the current stock prices for META and TESLA, as well as their stock prices at the beginning of the year (January 1, 2024). To do this, I will use the `yfinance` library to get the stock data.

I will write a Python script to perform these tasks. Please ensure you have the `yfinance` library installed. If you don't have it installed, you can do so by running `pip install yfinance`.

Here is the code to fetch the required stock prices and calculate the year-to-date gains:

```python

# filename: stock_gain_comparison.py
import yfinance as yf
from datetime import datetime

# Define the stock symbols
stocks = ['META', 'TSLA']

# Get today's date
today = datetime.now().date()

# Fetch current prices and prices at the beginning of the year
current_prices = {}
start_of_year_prices = {}

for stock in stocks:
    stock_data = yf.Ticker(stock)
    current_prices[stock] = stock_data.history(period='1d'
['Close'].iloc[-1]

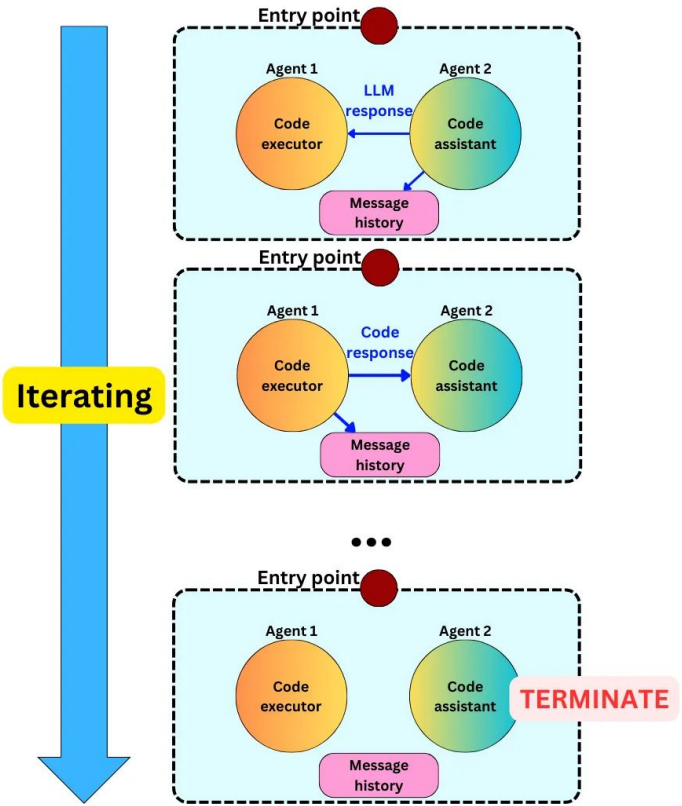
    start_of_year_prices[stock] = stock_data.history(start='2024-01-
01', end='2024-01-02')['Close'].iloc[0]

# Calculate year-to-date gains
ytd_gains = {stock: (current_prices[stock] -
start_of_year_prices[stock]) / start_of_year_prices[stock] * 100 for
stock in stocks}

# Print the results
print(f"Current Prices: {current_prices}")
print(f"Start of Year Prices: {start_of_year_prices}")
print(f"Year-to-Date Gains: {ytd_gains}")
```

Please save this code in a file named `stock_gain_comparison.py` and execute it.

2-agents conversation/collaboration (cont'd)



The execution was successful, and here are the results:

- ***Current Prices:***
  - *META: $536.32*
  - *TESLA: $227.87*
- ***Start of Year Prices:***
  - *META: $345.25*
  - *TESLA: $248.42*
- ***Year-to-Date Gains:***
  - *META: **55.34%***
  - *TESLA: **-8.27%***

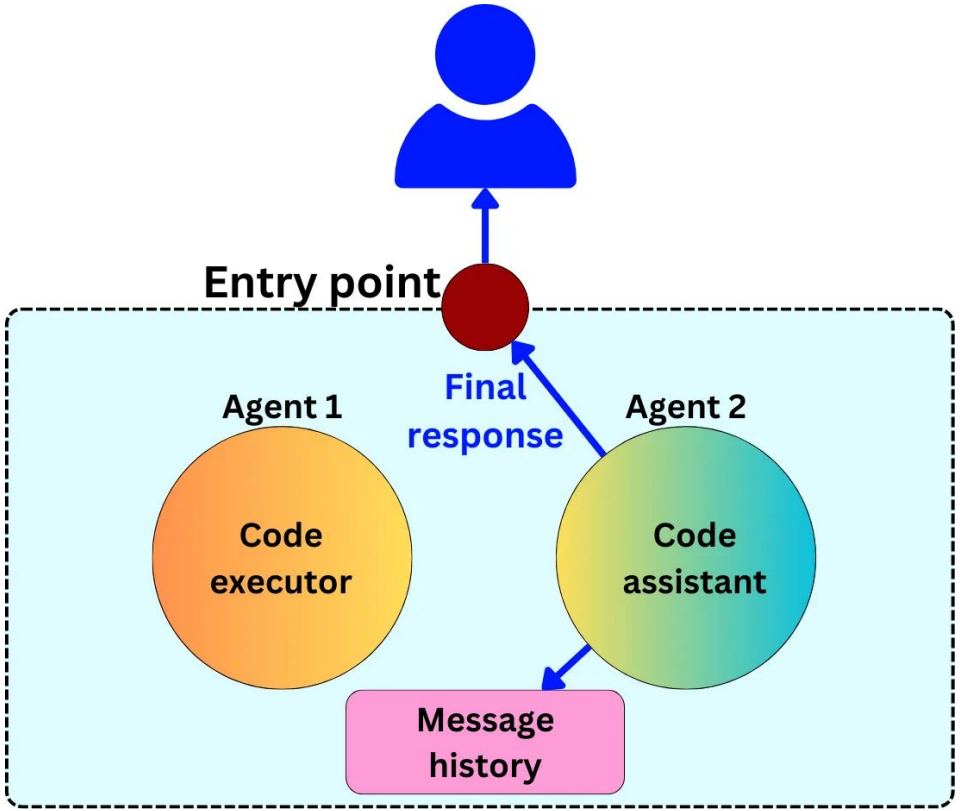
This indicates that META has gained approximately 55.34% since the beginning of the year, while TESLA has decreased by about 8.27%.

If you need any further analysis or information, please let me know.

TERMINATE

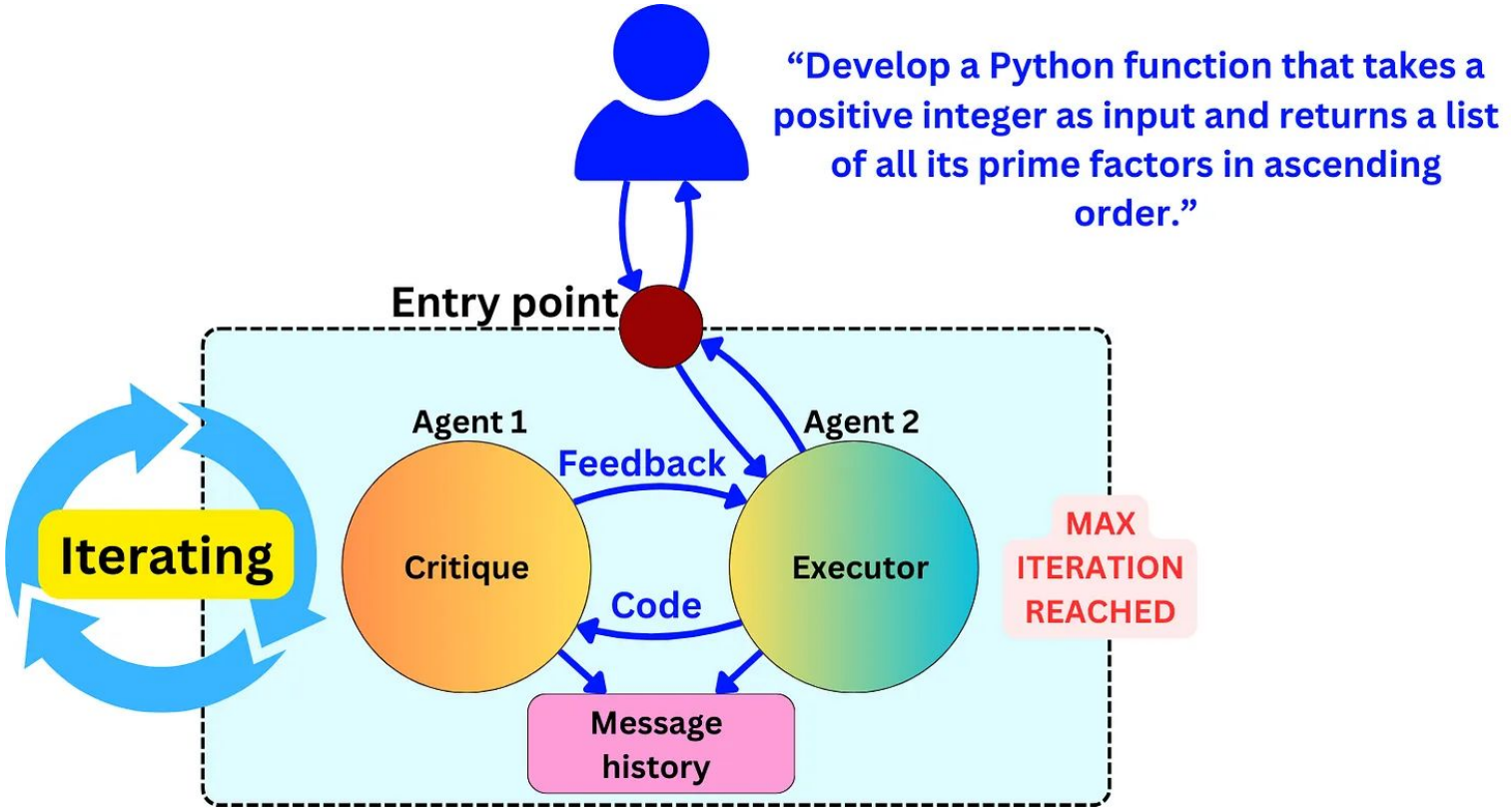
"The Different Agentic Patterns" by Damien Benveniste

2-agents conversation/collaboration (cont'd)



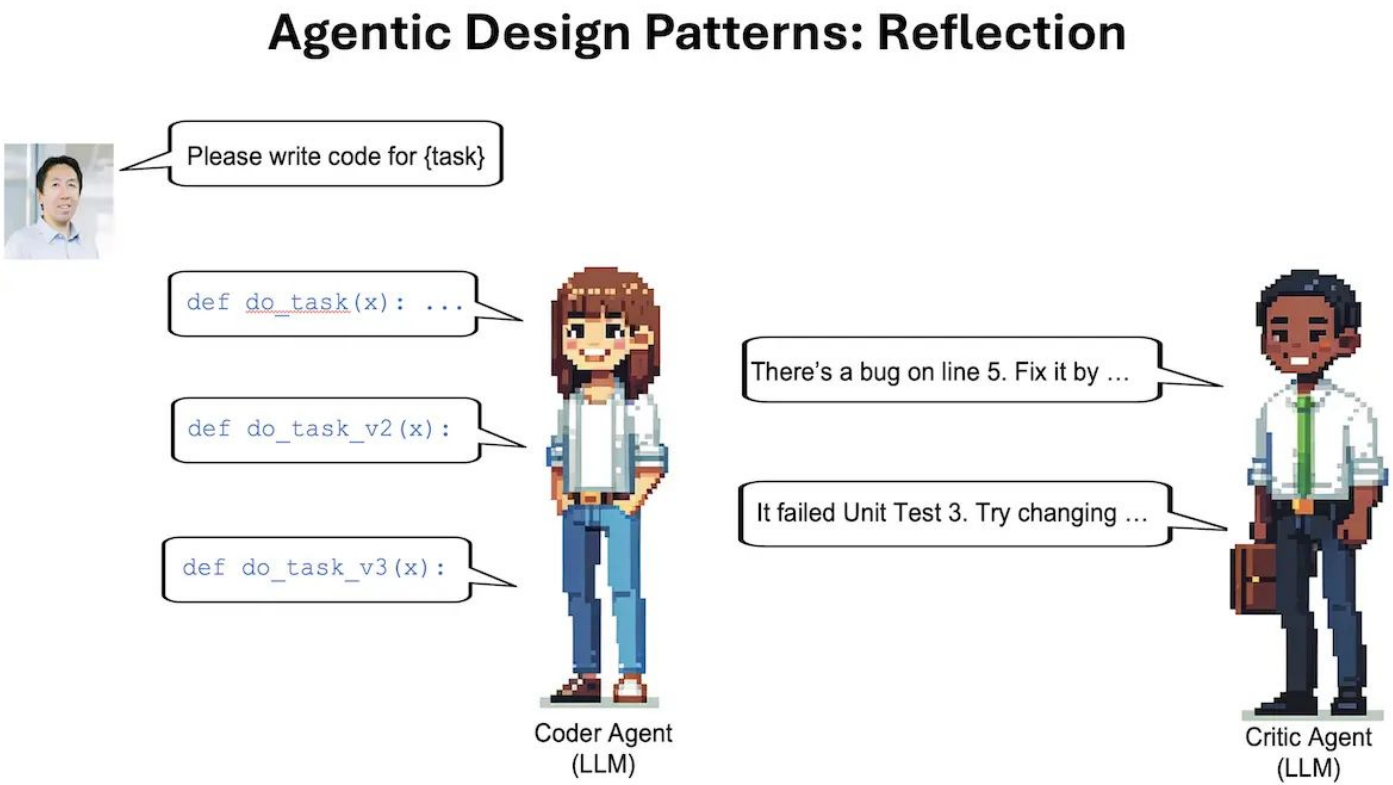
"The Different Agentic Patterns" by Damien Benveniste

Reflection



"The Different Agentic Patterns" by Damien Benveniste

Reflection (cont'd)



https://www.deeplearning.ai/the-batch/agentic-design-patterns-part-2-reflection

Reflection (cont'd)

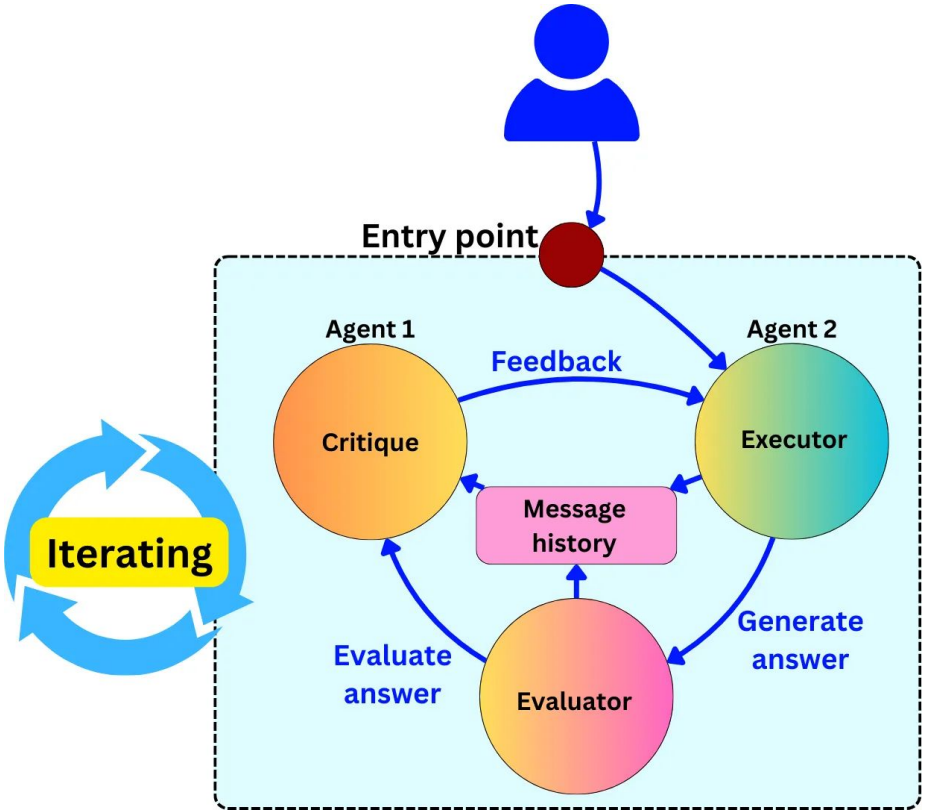
> ***"Your task is to write Python code that solves the problem presented to you. Ensure that the code is correct, efficient, well-structured, and handles edge cases. Comment the code where necessary and ensure it's readable and maintainable.***
>
> ***If the user provides critique, respond with a revised version of your previous attempts."***

And for the critique

> ***"Your task is to review and critique the code generated by the Executor Agent. Check for correctness, efficiency, code clarity, and completeness. Identify any potential issues, such as missed edge cases, inefficiencies, or areas where the code could be made more readable. Provide constructive feedback and suggest improvements."***

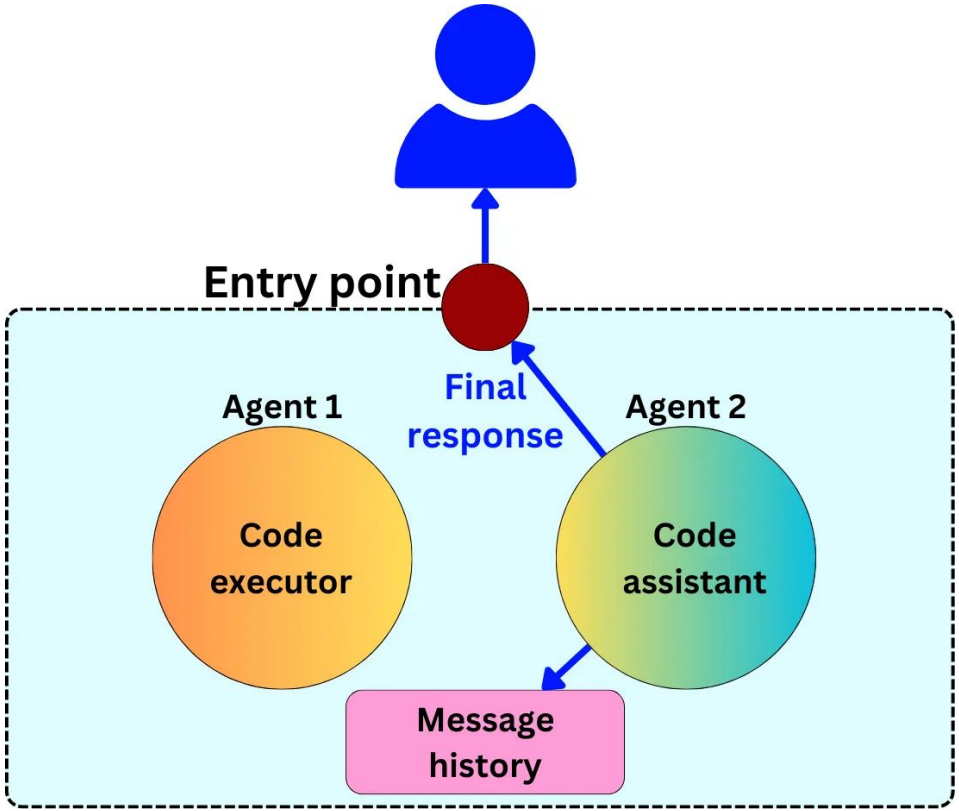
"The Different Agentic Patterns" by Damien Benveniste

Reflection (cont'd)



"The Different Agentic Patterns" by Damien Benveniste

Tool use



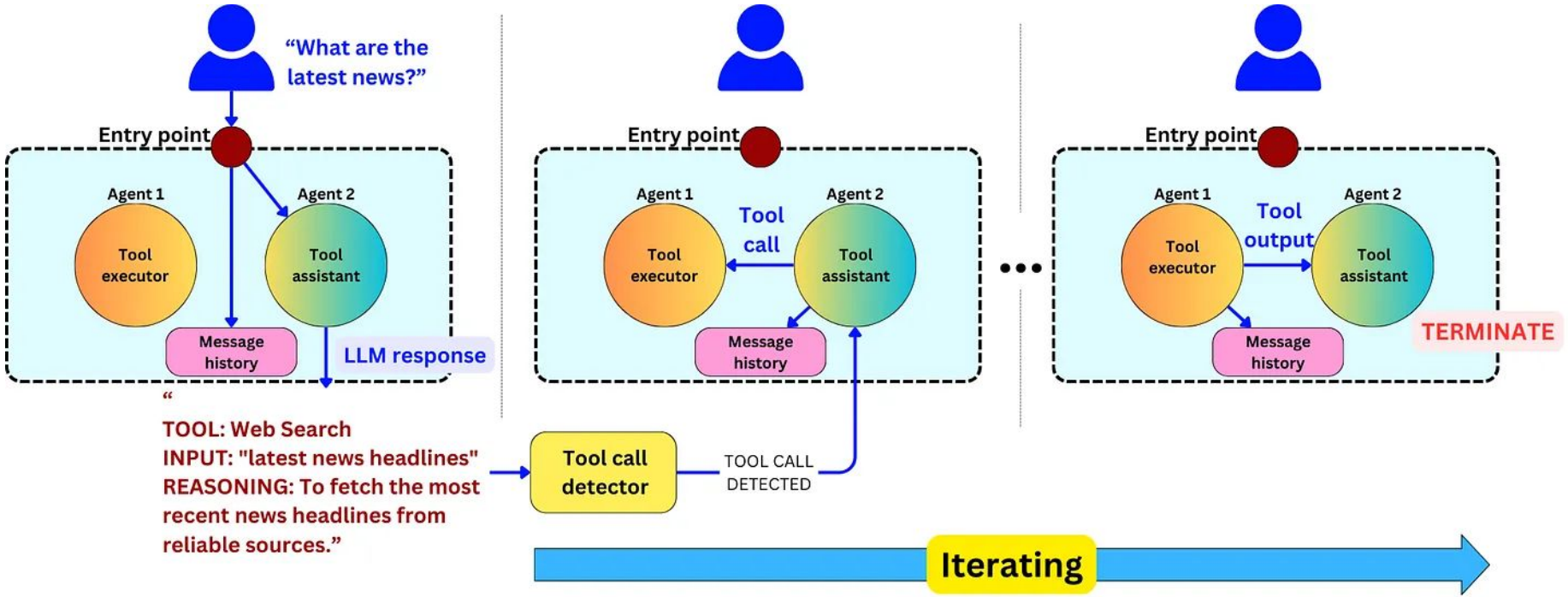
"The Different Agentic Patterns" by Damien Benveniste

System prompt for the code assistant

> *You are an assistant with access to two tools: Web Search and Calculator. When using a tool, structure your response as follows:*
>
> - *TOOL: [Tool name]*
> - *INPUT: [Tool input]*
> - *REASONING: [Brief explanation]*
>
> *Always end your response with:*
>
> *ANSWER: [Your final response to the query]*
>
> *Only use tools when necessary. If no tool is needed, just provide the ANSWER.*

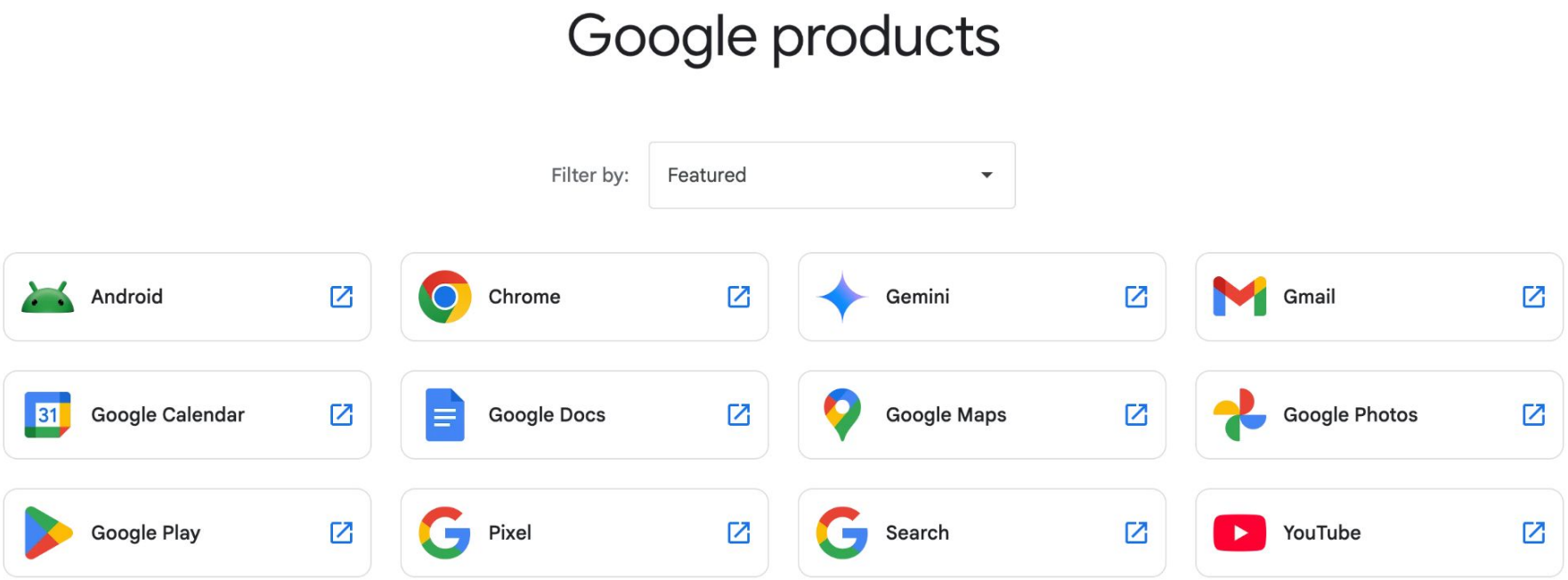
"The Different Agentic Patterns" by Damien Benveniste

Tool use

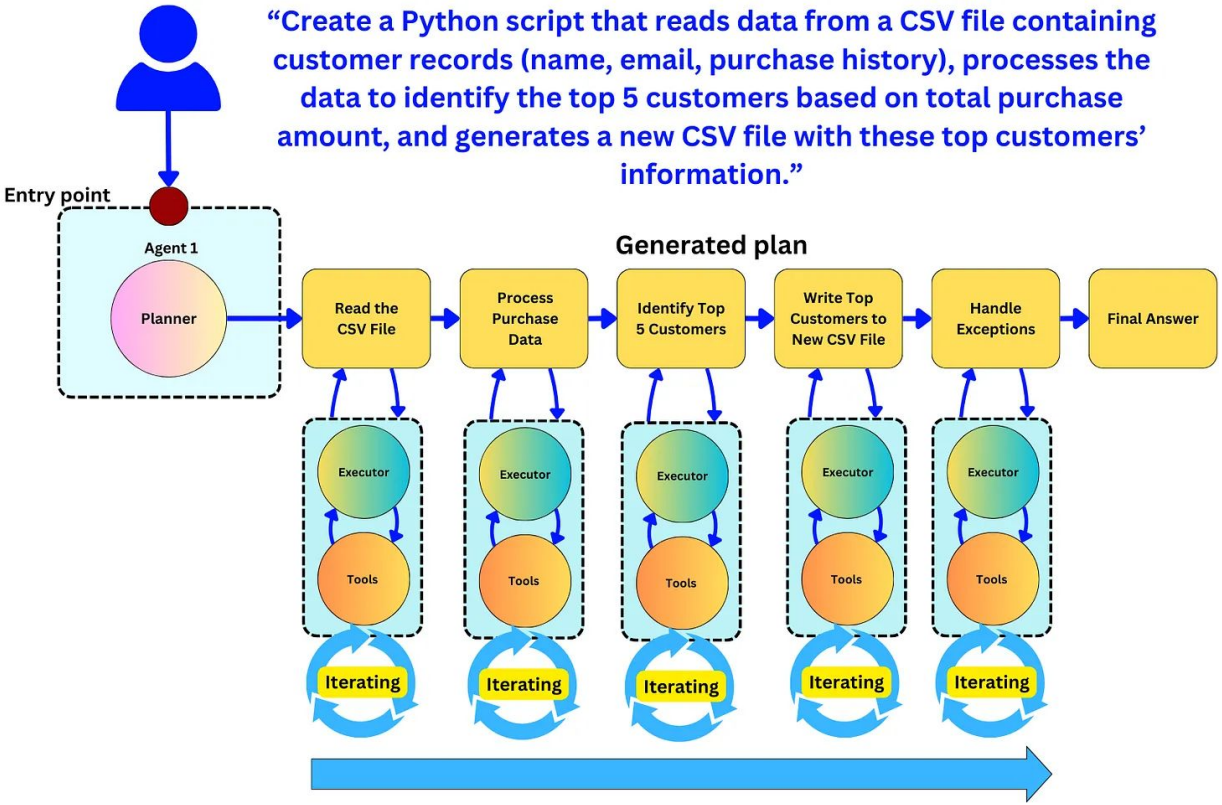


"The Different Agentic Patterns" by Damien Benveniste

When we have many tools

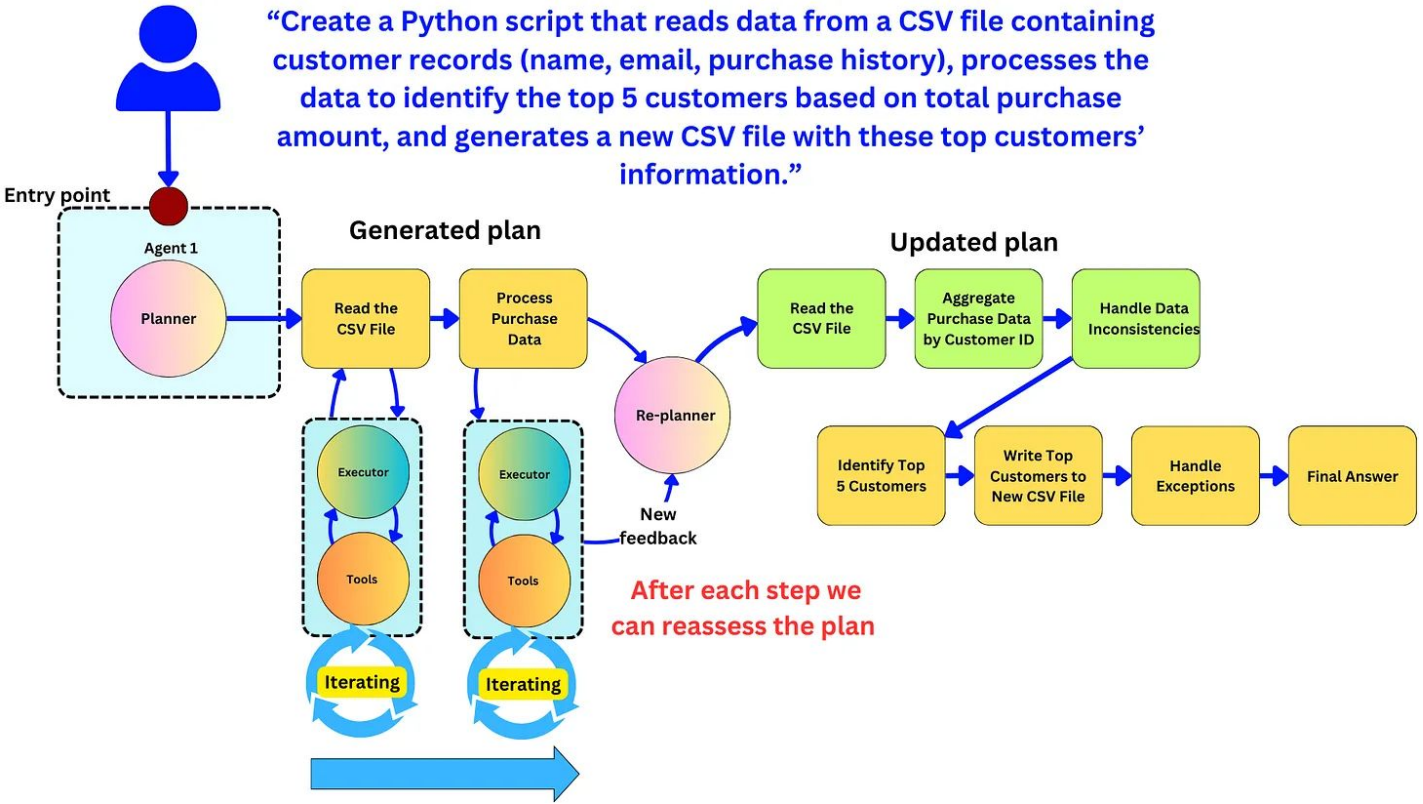


Planning



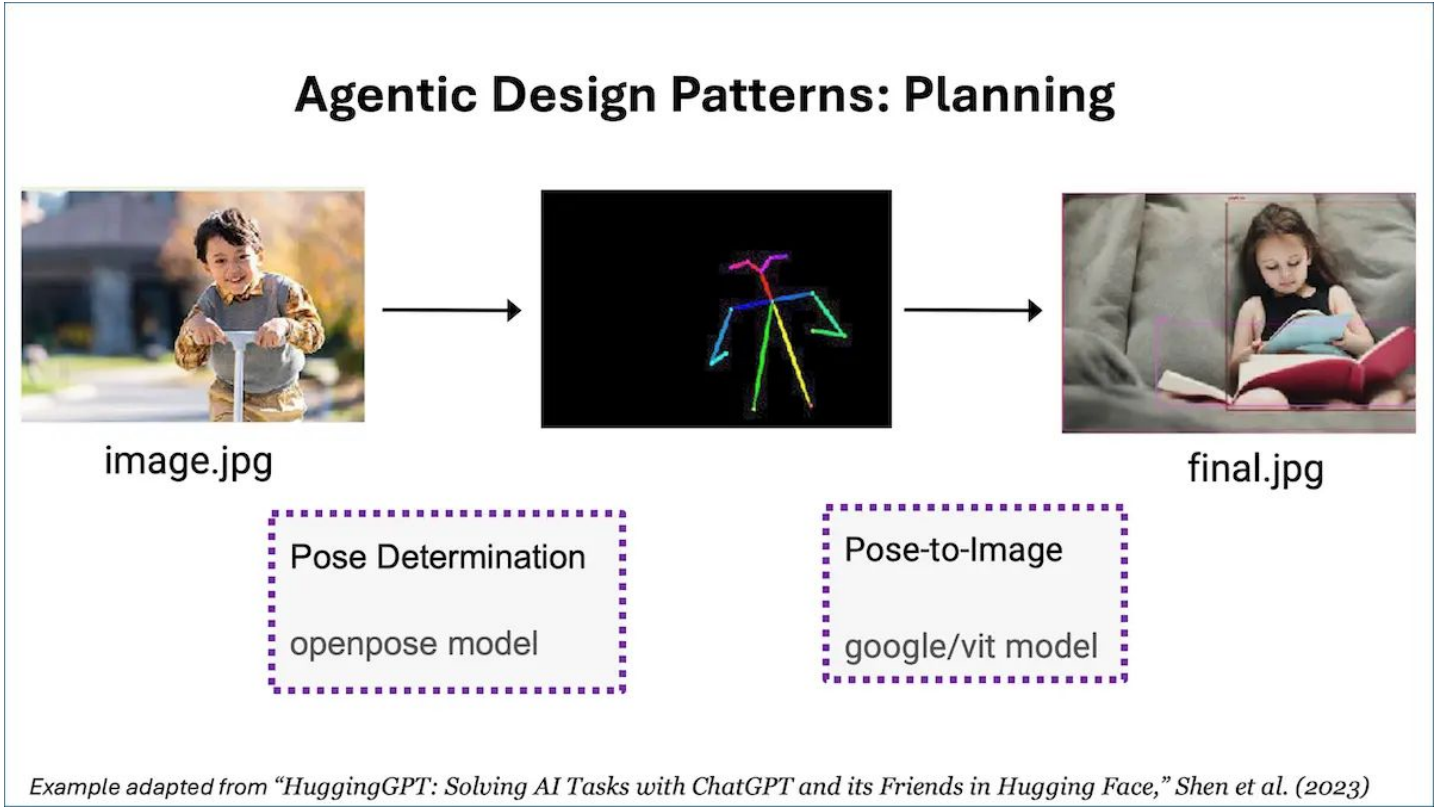
"The Different Agentic Patterns" by Damien Benveniste

Planning (cont'd)



"The Different Agentic Patterns" by Damien Benveniste

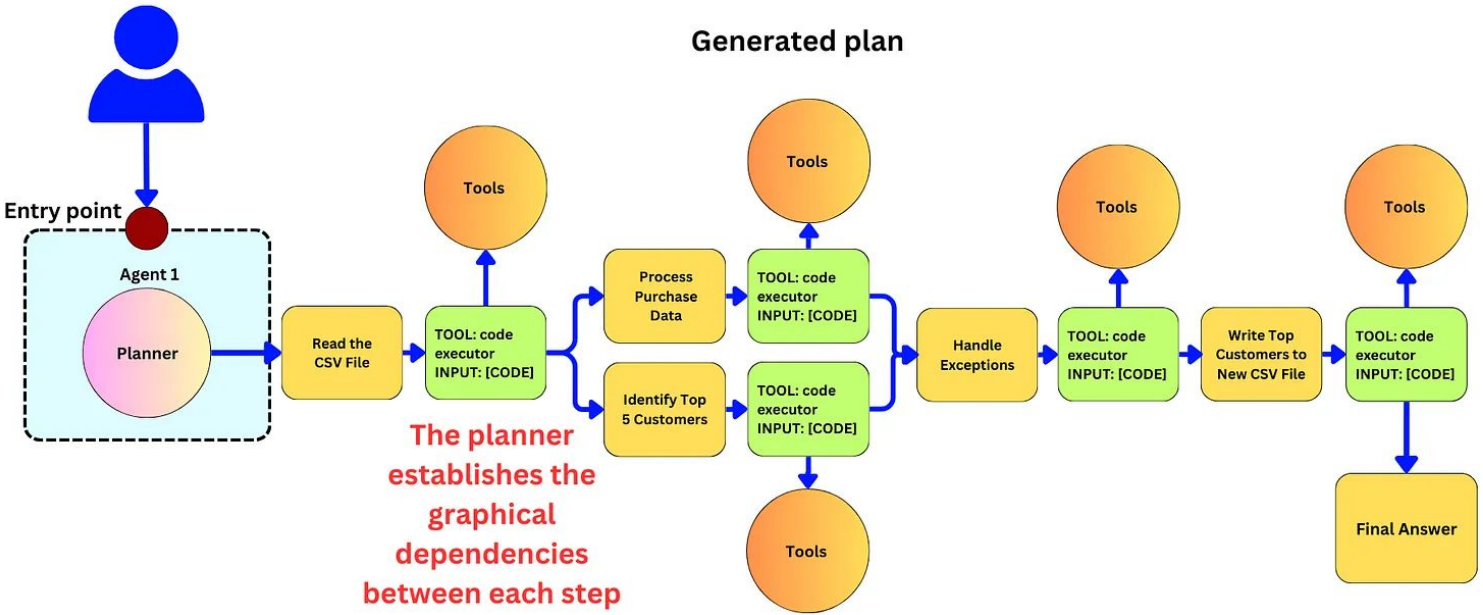
Planning (cont'd)



https://www.deeplearning.ai/the-batch/agentic-design-patterns-part-4-planning

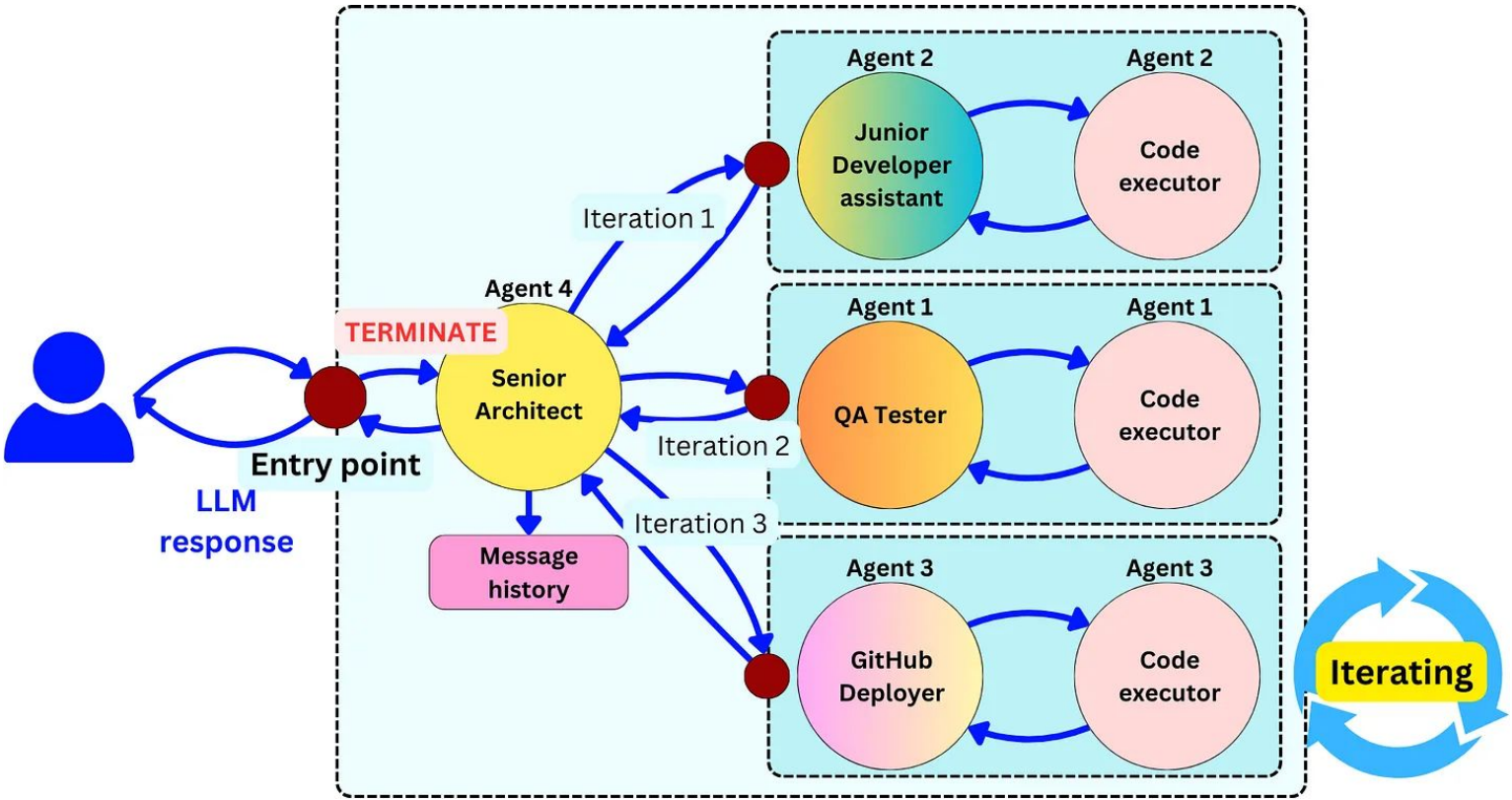
Planning (cont'd)

"Create a Python script that reads data from a CSV file containing customer records (name, email, purchase history), processes the data to identify the top 5 customers based on total purchase amount, and generates a new CSV file with these top customers' information."



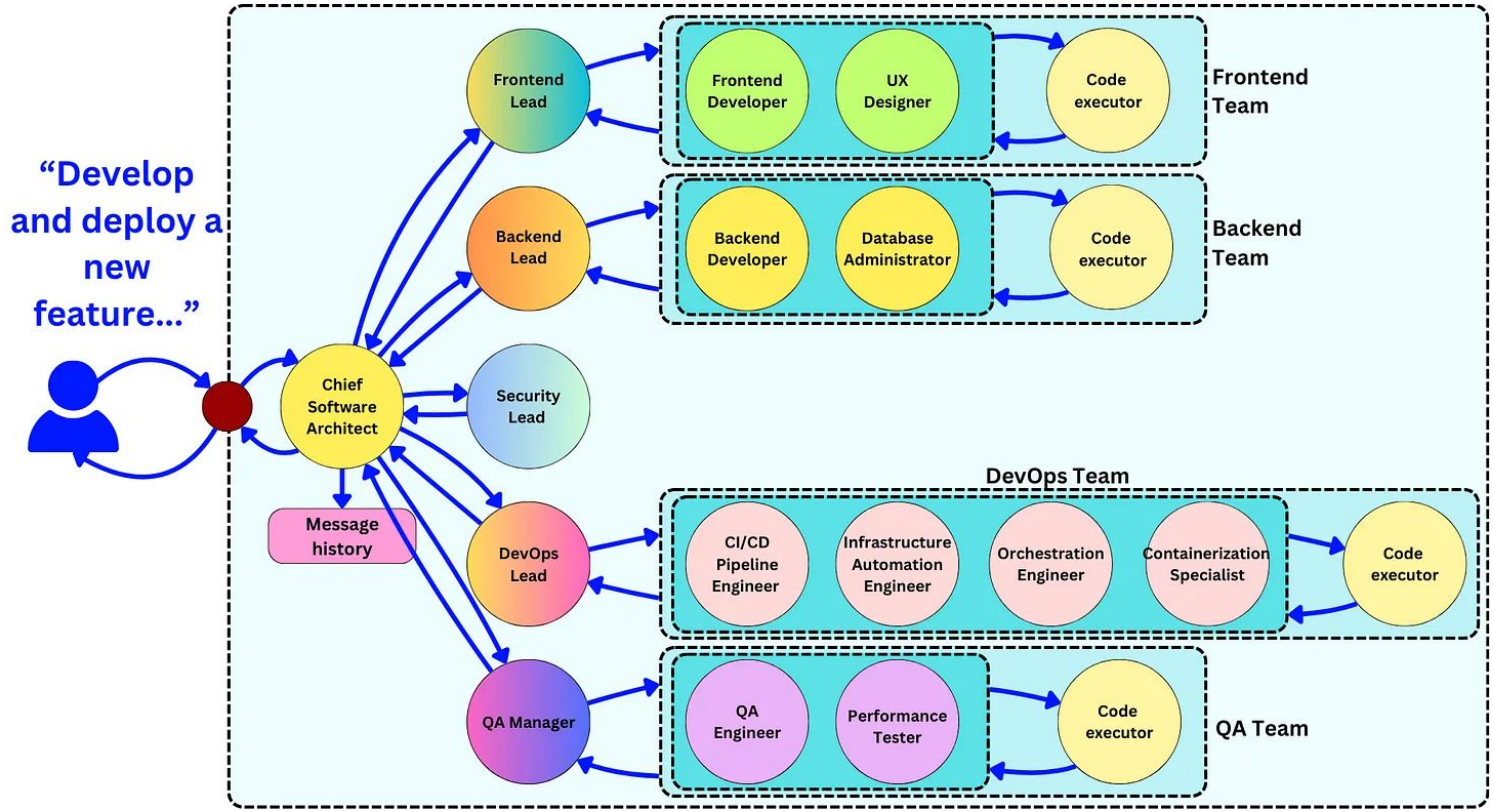
"The Different Agentic Patterns" by Damien Benveniste

Multi-agent collaboration



"The Different Agentic Patterns" by Damien Benveniste

Multi-agent collaboration (cont'd)



"The Different Agentic Patterns" by Damien Benveniste

Popular agent use cases

- Coding agents
- Computer-use agents
- Deep research / Scientific Agents

Coding agents: Cursor demo

Observability & Tracing
Secret Registry
LLM Features
Agent Features
Conversation Features
Remote Agent Server
GitHub Workflows

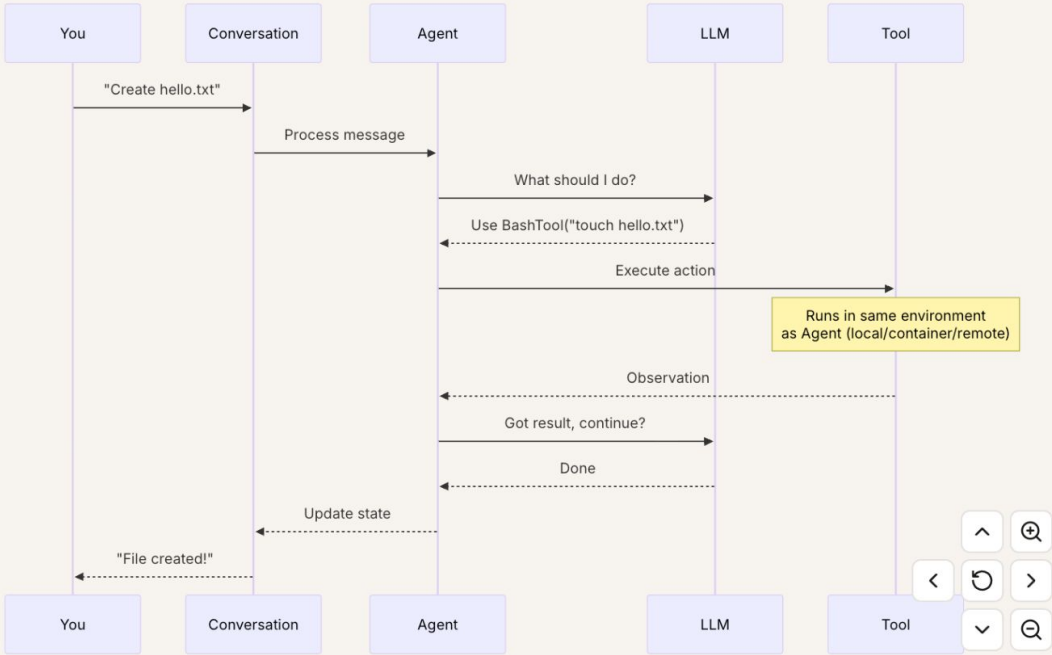
Architecture
Overview
Design Principles
SDK Components

API Reference
openhands.sdk.agent
openhands.sdk.conversation

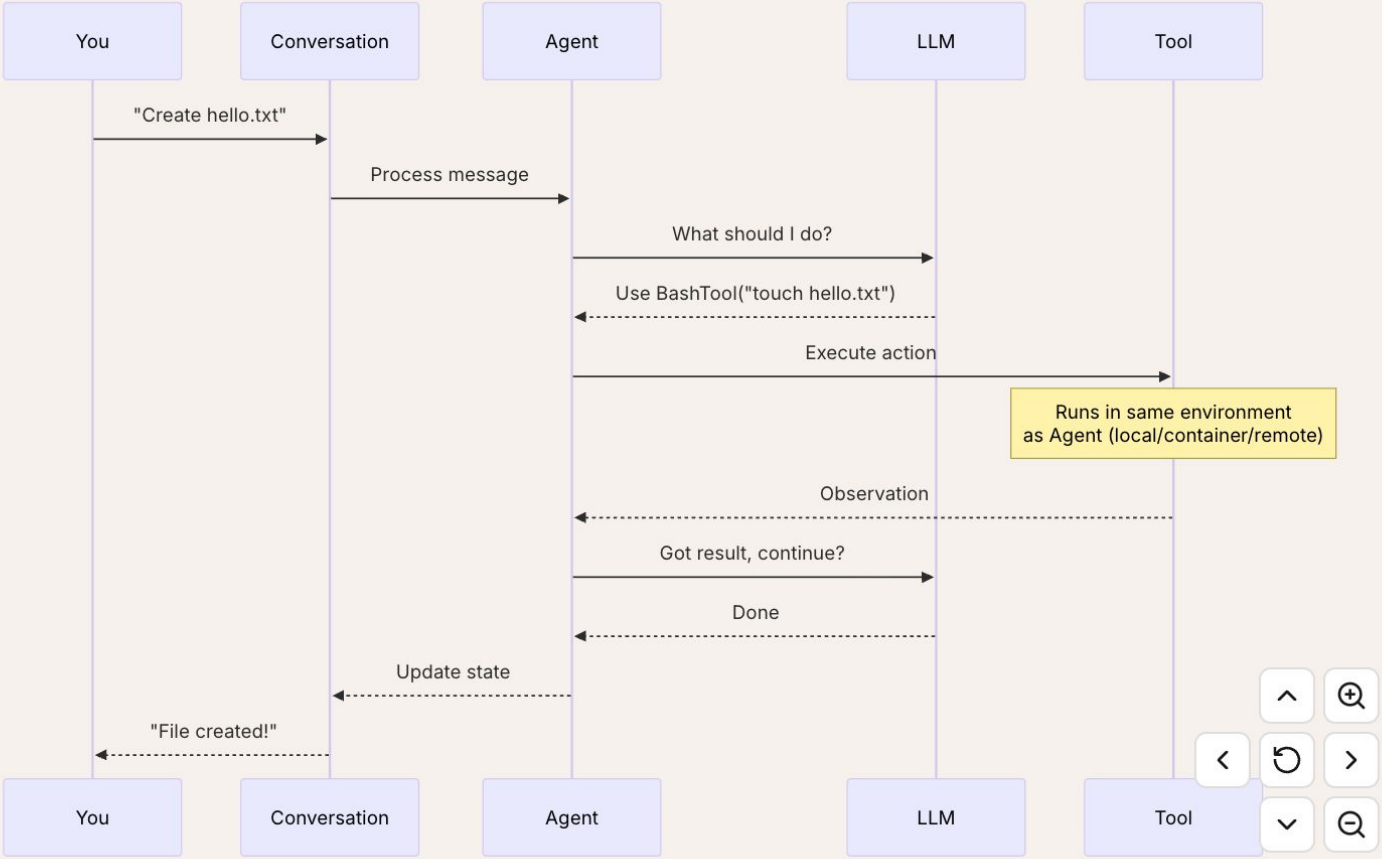
How Components Work Together

Basic Execution Flow (Local)

When you send a message to an agent, here's what happens:

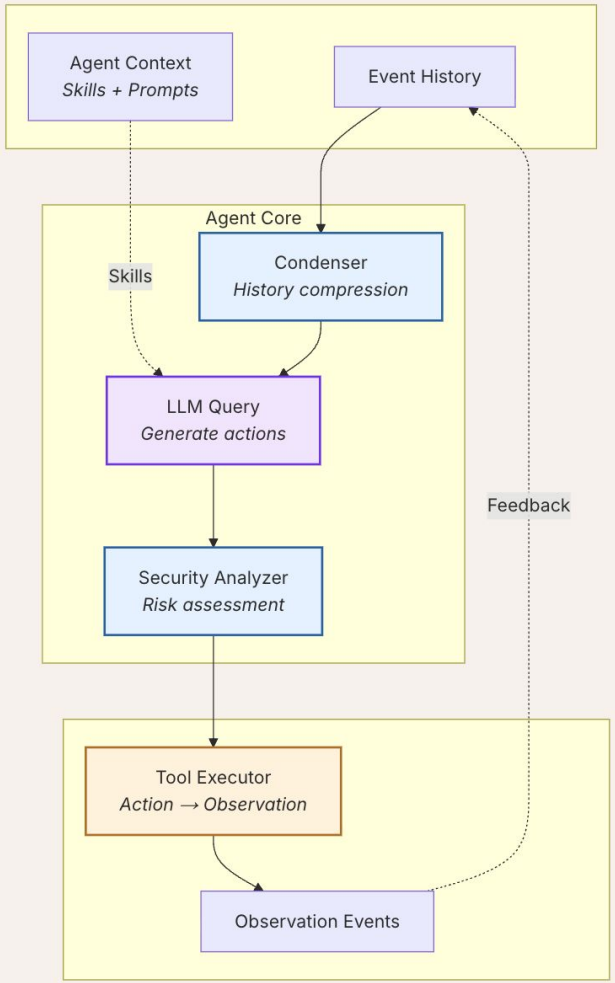


When you send a message to an agent, here's what happens:



https://docs.openhands.dev/sdk/arch/overview

Architecture



https://docs.openhands.dev/sdk/arch/agent

Tools

https://github.com/OpenHands/software-agent-sdk/tree/main/openhands-tools/openhands/tools

Computer use agents

OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments

Tianbao Xie[1], Danyang Zhang[1], Jixuan Chen[1], Xiaochuan Li[1],
Siheng Zhao[1], Ruisheng Cao[1], Toh Jing Hua[1], Zhoujun Cheng[1], Dongchan Shin[1], Fangyu Lei[1], Yitao Liu[1],
Yiheng Xu[1], Shuyan Zhou[3], Silvio Savarese[2], Caiming Xiong[2], Victor Zhong[4], Tao Yu[1]
[1]The University of Hong Kong, [2]Salesforce Research, [3]Carnegie Mellon University, [4]University of Waterloo

2025-07-28: Major Upgrade! OSWorld has been enhanced and is now OSWorld-Verified with comprehensive improvements: fixed community-reported examples, AWS support reducing evaluation time to within 1 hour, and updated benchmark results. See the verified benchmark results in the Benchmark section below. Please compare your OSWorld results with the new benchmark results when running the latest version.

Paper | Code | Doc | Data | Data Viewer | Slides | Twitter | Discord

Data viewer

https://os-world.github.io/explorer.html

ChatGPT Agents demo

In the game Pizza Car on CrazyGames, the player is first told to follow the arrows. If the player instead turns backwards and follows the right lanes of the road in front of them, how many street lights do they pass before reaching the first intersection?

arXiv:2504.00906v1 [cs.AI] 1 Apr 2025

Agent S2: A Compositional Generalist-Specialist Framework for Computer Use Agents

Saaket Agashe,* **Kyle Wong,*** **Vincent Tu,*** **Jiachen Yang, Ang Li, Xin Eric Wang**
Simular Research

Abstract

Computer use agents automate digital tasks by directly interacting with graphical user interfaces (GUIs) on computers and mobile devices, offering significant potential to enhance human productivity by completing an open-ended space of user queries. However, current agents face significant challenges: imprecise grounding of GUI elements, difficulties with long-horizon task planning, and performance bottlenecks from relying on single generalist models for diverse cognitive tasks. To this end, we introduce Agent S2, a novel compositional framework that delegates cognitive responsibilities across various generalist and specialist models. We propose a novel Mixture-of-Grounding technique to achieve precise GUI localization and introduce Proactive Hierarchical Planning, dynamically refining action plans at multiple temporal scales in response to evolving observations. Evaluations demonstrate that Agent S2 establishes new state-of-the-art (SOTA) performance on three prominent computer use benchmarks. Specifically, Agent S2 achieves 18.9% and 32.7% relative improvements over leading baseline agents such as Claude Computer Use and UI-TARS on the OSWorld 15-step and 50-step evaluation. Moreover, Agent S2 generalizes effectively to other operating systems and applications, surpassing previous best methods by 52.8% on WindowsAgentArena and by 16.52% on AndroidWorld relatively. Code available at https://github.com/simular-ai/Agent-S.

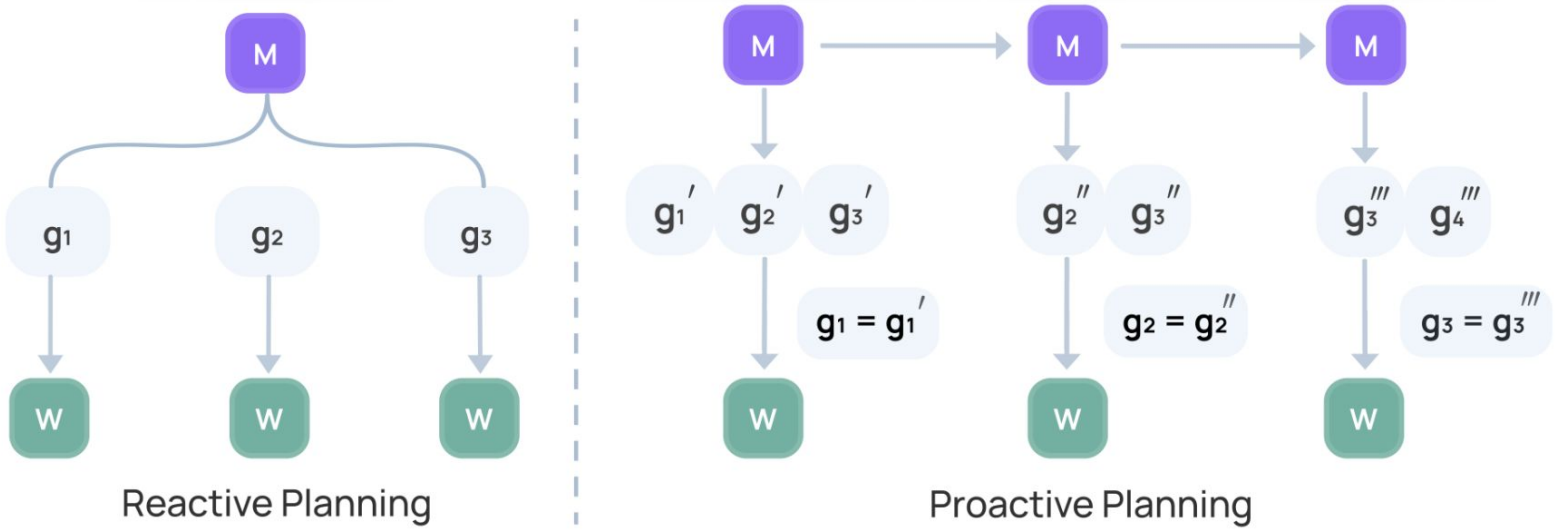


Figure 3: Comparison between Reactive and Proactive Planning. Proactive planning re-evaluates and updates the remainder of the plan after every subtask, while reactive planning adheres to a fixed plan and only revises it in response to subtask failures.

5.18096v2 [cs.AI] 3 Sep 2025

DEEP RESEARCH AGENTS: A SYSTEMATIC EXAMINATION AND ROADMAP

Yuxuan Huang[†1], Yihang Chen[†2], Haozheng Zhang[†2], Kang Li[3], Huichi Zhou[4], Meng Fang[1], Linyi Yang[4], Xiaoguang Li[2], Lifeng Shang[2], Songcen Xu[2], Jianye Hao[2], Kun Shao[‡2], and Jun Wang[‡4]

[1] University of Liverpool [2] Huawei Noah's Ark Lab [3] University of Oxford [4] University College London

ABSTRACT

The rapid progress of Large Language Models (LLMs) has given rise to a new category of autonomous AI systems, referred to as Deep Research (DR) agents. These agents are designed to tackle complex, multi-turn informational research tasks by leveraging a combination of dynamic reasoning, adaptive long-horizon planning, multi-hop information retrieval, iterative tool use, and the generation of structured analytical reports. In this paper, we conduct a detailed analysis of the foundational technologies and architectural components that constitute Deep Research agents. We begin by reviewing information acquisition strategies, contrasting API-based retrieval methods with browser-based exploration. We then examine modular tool-use frameworks, including code execution, multimodal input processing, and the integration of Model Context Protocols (MCPs) to support extensibility and ecosystem development. To systematise existing approaches, we propose a taxonomy that differentiates between static and dynamic workflows, and we classify agent architectures based on planning strategies and agent composition, including single-agent and multi-agent configurations. We also provide a critical evaluation of current benchmarks, highlighting key limitations such as restricted access to external knowledge, sequential execution inefficiencies, and misalignment between evaluation metrics and the practical objectives of DR agents. Finally, we outline open challenges and promising directions for future research. A curated and continuously updated repository of DR agent research is available at: https://github.com/ai-agents-2030/awesome-deep-research-agent.

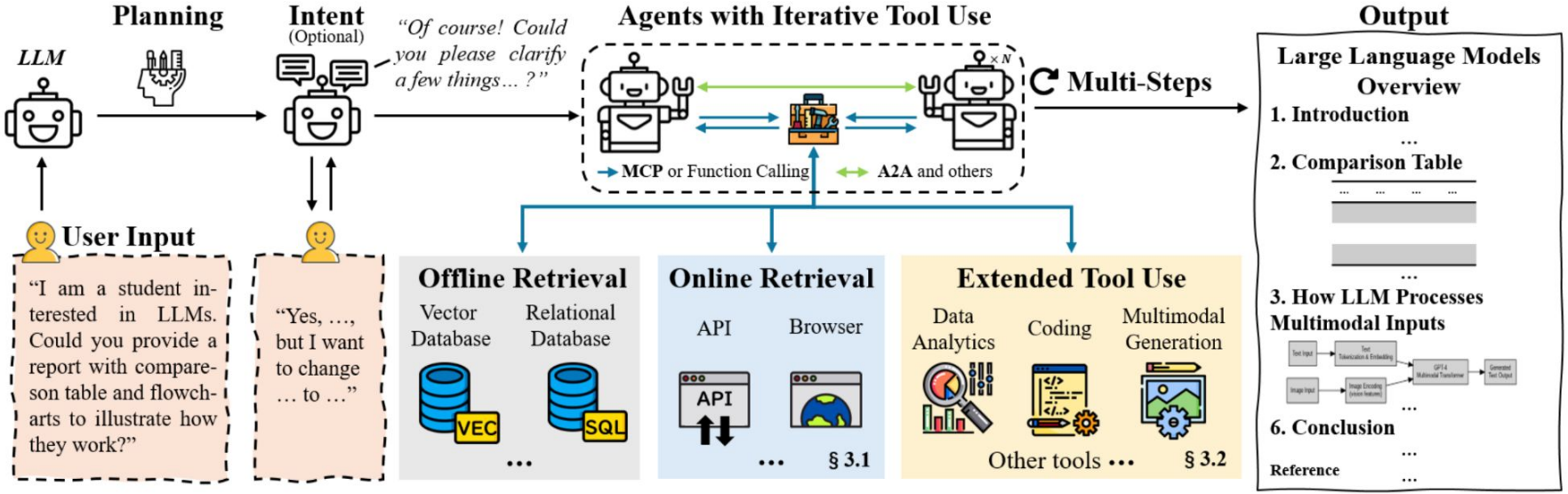


Figure 1: A structural overview of a DR agent in a multi-agent architecture for ease of illustration.

Deep Research demo

Synthetic Data Generation & Multi-Step RL for Reasoning & Tool Use

Anna Goldie*[1] Azalia Mirhoseini*[1] Hao Zhou[2] Irene Cai[2] Christopher D. Manning[1]

[1] Department of Computer Science, Stanford University
[2] Google DeepMind
* Equal contribution
{agoldie,azalia}@cs.stanford.edu

arXiv:2504.04736v2 [cs.AI] 28 Apr 2025

Abstract

Reinforcement learning has been shown to improve the performance of large language models. However, traditional approaches like RLHF or RLAIF treat the problem as single-step. As focus shifts toward more complex reasoning and agentic tasks, language models must take multiple steps of text generation, reasoning and environment interaction before generating a solution. We propose a synthetic data generation and RL methodology targeting multi-step optimization scenarios. This approach, called Step-Wise Reinforcement Learning (SWiRL), iteratively generates multi-step reasoning and tool use data, and then learns from that data. It employs a simple step-wise decomposition that breaks each multi-step trajectory into multiple sub-trajectories corresponding to each action by the original model. It then applies synthetic data filtering and RL optimization on these sub-trajectories. We evaluated SWiRL on a number of multi-step tool use, question answering, and mathematical reasoning tasks. Our experiments show that SWiRL outperforms baseline approaches by 21.5%, 12.3%, 14.8%, 11.1%, and 15.3% in relative accuracy on GSM8K, HotPotQA, CofCA, MuSiQue, and BeerQA, respectively. Excitingly, the approach exhibits generalization across tasks: for example, training only on HotPotQA (text question-answering) improves zero-shot performance on GSM8K (a math dataset) by a relative 16.9%.

Thank you!